

Medical Data Standardisation

Complete Project Source Code Compilation

Generated: 2026-07-04

Target Folders: config, database, src, tests, ui

Folder: config

File: analytics_rules.yaml

Path: config\analytics_rules.yaml

```
non_numeric_tests:
  - WIDAL
  - HIV
  - COVID19
  - CRP_QUALITATIVE

allowed_values:
  - Positive
  - Negative
  - Reactive
  - Non Reactive
  - Non-Reactive
  - "1:20"
  - "1:40"
  - "1:80"
  - "1:160"
  - "1:320"

missing_values:
  - ""
  - "null"
  - "N/A"
  - "NA"
  - "none"
```

File: field_aliases.yaml

Path: config\field_aliases.yaml

```
patient_name:
  - patientName
  - patient_name
  - patient
  - name
  - patientFullName
  - patient_full_name
  - basic_info.patient_name

age:
  - age
  - patientAge
  - patient_age
  - age_years
  - basic_info.age

gender:
  - gender
  - sex
  - patientGender
  - patient_gender

hospital_name:
  - hospitalName
  - hospital
  - hospital_name
  - lab_or_hospital_name
  - basic_info.lab_or_hospital_name

doctor_name:
  - doctorName
  - doctor
  - consultant
  - basic_info.doctor_name
  - attending_doctor
```

File: medicine_dictionary.yaml

Path: config\medicine_dictionary.yaml

```
MISOPROSTOL:
  aliases:
    - Tab. miso
    - TAB MISO
    - Misoprostol
    - MISOPROSTOL
    - induction dose
  canonical: Misoprostol
  type: Tablet
```

```
PARACETAMOL:
  aliases:
    - Tab. Paracetamol
    - TAB PARACETAMOL
    - Paracetamol
    - Crocin
    - Calpol
    - PCM
  canonical: Paracetamol
  type: Tablet
```

```
AMOXICILLIN:
  aliases:
    - Amoxycillin
    - Amoxicillin
    - Cap. Amox
    - Mox
  canonical: Amoxicillin
  type: Capsule
```

File: reference_ranges.yaml

Path: config/reference_ranges.yaml

```
# Numerical Reference Ranges & Thresholds
# Standard Units: Hemoglobin (g/dL), Creatinine (mg/dL), Electrolytes (mmol/L),
# WBC/RBC/Platelets (cells/uL or count/uL)

HAEMOGLOBIN:
  unit: g/dL
  low: 13.0
  high: 17.0
  outlier_low: 5.0
  outlier_high: 25.0
  Male:
    low: 13.0
    high: 17.0
  Female:
    low: 12.0
    high: 15.0
  Child:
    low: 10.0
    high: 14.0

WHITE_BLOOD_CELL_COUNT:
  unit: cells/cu.mm
  low: 4000.0
  high: 11000.0
  outlier_low: 500.0
  outlier_high: 50000.0

SERUM_CREATININE:
  unit: mg/dL
  low: 0.66
  high: 1.25
  outlier_low: 0.1
  outlier_high: 10.0

SODIUM:
  unit: mmol/L
  low: 135.0
  high: 145.0
  outlier_low: 100.0
  outlier_high: 180.0

POTASSIUM:
  unit: mmol/L
  low: 3.5
  high: 5.0
  outlier_low: 1.5
  outlier_high: 10.0
```

RBC_COUNT:

unit: Million/cu.mm
low: 4.5
high: 5.5
outlier_low: 1.5
outlier_high: 8.0

PLATELET_COUNT:

unit: /cu.mm
low: 150000.0
high: 410000.0
outlier_low: 10000.0
outlier_high: 1000000.0

NEUTROPHILS:

unit: "%"
low: 40.0
high: 80.0
outlier_low: 5.0
outlier_high: 95.0

LYMPHOCYTES:

unit: "%"
low: 20.0
high: 40.0
outlier_low: 5.0
outlier_high: 95.0

PACKED_CELL_VOLUME:

unit: "%"
low: 40.0
high: 50.0
outlier_low: 10.0
outlier_high: 70.0

MEAN_CORPUSCULAR_VOLUME:

unit: fL
low: 83.0
high: 101.0
outlier_low: 50.0
outlier_high: 130.0

MCH:

unit: pg
low: 27.0
high: 32.0
outlier_low: 15.0
outlier_high: 50.0

MCHC:

unit: g/dL
low: 31.5
high: 34.5

```
outlier_low: 20.0
outlier_high: 45.0

RDW:
  unit: "%"
  low: 11.6
  high: 14.0
  outlier_low: 5.0
  outlier_high: 30.0

SGOT:
  unit: U/L
  low: 17.0
  high: 59.0
  outlier_low: 2.0
  outlier_high: 500.0

SGPT:
  unit: U/L
  low: 21.0
  high: 72.0
  outlier_low: 2.0
  outlier_high: 500.0

# Qualitative Tests
COVID19:
  validation_type: qualitative
  allowed:
    - Positive
    - Negative
    - Reactive
    - Non Reactive
    - Non-Reactive

HIV:
  validation_type: qualitative
  allowed:
    - Positive
    - Negative
    - Reactive
    - Non Reactive
    - Non-Reactive

WIDAL:
  validation_type: qualitative
  allowed:
    - Positive
    - Negative
    - Reactive
    - Non Reactive
    - Non-Reactive
    - 1:20
    - 1:40
```


- 1:80
- 1:160

File: test_name_dictionary.yaml

Path: config\test_name_dictionary.yaml

```
HAEMOGLOBIN:
  canonical: HAEMOGLOBIN
  aliases:
    - Hb
    - HGB
    - Haemoglobin
    - aemoglobin
    - Hemoglobin
  category: Hematology
  expected_unit: g/dL
  expected_type: numeric

WHITE_BLOOD_CELL_COUNT:
  canonical: WHITE_BLOOD_CELL_COUNT
  aliases:
    - WBC
    - WBC Count
    - White Blood Cells
    - Total Leucocyte Count
    - TLC
  category: Hematology
  expected_unit: cells/cu.mm
  expected_type: numeric

SERUM_CREATININE:
  canonical: SERUM_CREATININE
  aliases:
    - Creatinine
    - Serum Creatinine
    - Creat
    - S. Creatinine
  category: Biochemistry
  expected_unit: mg/dL
  expected_type: numeric

SODIUM:
  canonical: SODIUM
  aliases:
    - Sodium
    - Na
    - Na+
    - Serum Sodium
  category: Electrolytes
  expected_unit: mmol/L
  expected_type: numeric

POTASSIUM:
```

```
canonical: POTASSIUM
aliases:
  - Potassium
  - K
  - K+
  - Serum Potassium
category: Electrolytes
expected_unit: mmol/L
expected_type: numeric
```

```
RBC_COUNT:
canonical: RBC_COUNT
aliases:
  - RBC
  - RBC Count
  - Red Blood Cells
  - Total RBC Count
category: Hematology
expected_unit: Million/cu.mm
expected_type: numeric
```

```
PLATELET_COUNT:
canonical: PLATELET_COUNT
aliases:
  - Platelet Count
  - Platelets
  - PLT
category: Hematology
expected_unit: /cu.mm
expected_type: numeric
```

```
NEUTROPHILS:
canonical: NEUTROPHILS
aliases:
  - Neutrophils
  - Neutro
  - Polymorphs
category: Hematology
expected_unit: "%"
expected_type: numeric
```

```
LYMPHOCYTES:
canonical: LYMPHOCYTES
aliases:
  - Lymphocytes
  - Lympho
category: Hematology
expected_unit: "%"
expected_type: numeric
```

```
PACKED_CELL_VOLUME:
canonical: PACKED_CELL_VOLUME
aliases:
  - Packed Cell Volume
```

```
- PCV
- Hematocrit
- HCT
category: Hematology
expected_unit: "%"
expected_type: numeric

MEAN_CORPUSCULAR_VOLUME:
canonical: MEAN_CORPUSCULAR_VOLUME
aliases:
  - Mean Corpuscular Volume
  - MCV
category: Hematology
expected_unit: fL
expected_type: numeric

MCH:
canonical: MCH
aliases:
  - MCH
  - Mean Corpuscular Hemoglobin
category: Hematology
expected_unit: pg
expected_type: numeric

MCHC:
canonical: MCHC
aliases:
  - MCHC
  - Mean Corpuscular Hemoglobin Concentration
category: Hematology
expected_unit: g/dL
expected_type: numeric

RDW:
canonical: RDW
aliases:
  - RDW
  - Red Cell Distribution Width
category: Hematology
expected_unit: "%"
expected_type: numeric

SGOT:
canonical: SGOT
aliases:
  - SGOT
  - AST
  - Aspartate Aminotransferase
category: Biochemistry
expected_unit: U/L
expected_type: numeric

SGPT:
```

```
canonical: SGPT
aliases:
  - SGPT
  - ALT
  - Alanine Aminotransferase
category: Biochemistry
expected_unit: U/L
expected_type: numeric
```

```
ALKALINE_PHOSPHATASE:
canonical: ALKALINE_PHOSPHATASE
aliases:
  - Alkaline Phosphatase
  - ALP
category: Biochemistry
expected_unit: U/L
expected_type: numeric
```

```
TOTAL_PROTEIN:
canonical: TOTAL_PROTEIN
aliases:
  - Total Protein
  - Protein
category: Biochemistry
expected_unit: g/dL
expected_type: numeric
```

```
ALBUMIN:
canonical: ALBUMIN
aliases:
  - Albumin
category: Biochemistry
expected_unit: g/dL
expected_type: numeric
```

```
GLOBULIN:
canonical: GLOBULIN
aliases:
  - Globulin
category: Biochemistry
expected_unit: g/dL
expected_type: numeric
```

```
AG_RATIO:
canonical: AG_RATIO
aliases:
  - A/G Ratio
  - Albumin/Globulin Ratio
category: Biochemistry
expected_unit: ratio
expected_type: numeric
```

```
COVID19:
canonical: COVID19
```

```
aliases:  
  - COVID-19  
  - SARS-CoV-2  
  - Corona Virus Test  
  - Corona  
category: Serology  
expected_unit: text  
expected_type: qualitative
```

```
HIV:  
  canonical: HIV  
  aliases:  
    - HIV I & II  
    - HIV Test  
    - Human Immunodeficiency Virus  
  category: Serology  
  expected_unit: text  
  expected_type: qualitative
```

```
WIDAL:  
  canonical: WIDAL  
  aliases:  
    - Widal  
    - Widal Test  
    - Typhoid Test  
  category: Serology  
  expected_unit: text  
  expected_type: qualitative
```

File: unit_conversion.yaml

Path: config\unit_conversion.yaml

```
# Unit Conversion & Scaling Factors
# All factors scale the original numeric value by multiplication (value_std = value_orig *
factor)

g/L:
  canonical: g/dL
  factor: 0.1

gm/dL:
  canonical: g/dL
  factor: 1.0

g/dL:
  canonical: g/dL
  factor: 1.0

mg/dL:
  canonical: mg/dL
  factor: 1.0

mg/l:
  canonical: mg/dL
  factor: 0.1

mmol/L:
  canonical: mmol/L
  factor: 1.0

mmol/l:
  canonical: mmol/L
  factor: 1.0

Million/cu.mm:
  canonical: Million/cu.mm
  factor: 1.0

mil/cu.mm:
  canonical: Million/cu.mm
  factor: 1.0

cells/cu.mm:
  canonical: cells/cu.mm
  factor: 1.0

/cu.mm:
  canonical: /cu.mm
  factor: 1.0
```

```
U/L:
  canonical: U/L
  factor: 1.0

u/l:
  canonical: U/L
  factor: 1.0

"%":
  canonical: "%"
  factor: 1.0

fL:
  canonical: fL
  factor: 1.0

fl:
  canonical: fL
  factor: 1.0

pg:
  canonical: pg
  factor: 1.0

ratio:
  canonical: ratio
  factor: 1.0
```


File: discharge_summary.yaml

Path: config\document_types\discharge_summary.yaml

```
document_type: discharge_summary
fields:
  patient_name: patientName
  doctor_name: doctorName
  diagnosis: diagnosis
  brief_history: briefHistory
  course_during_hospitalisation: courseDuringHospitalisation
  recommendations: recommendations
  hospital_address: hospitalAddress
  ward: ward
  age: age
  gender: gender
  medicines_array: dischargeMedications
```

File: lab_report.yaml

Path: config\document_types\lab_report.yaml

```
document_type: lab_report
fields:
  patient_name: basic_info.patient_name
  hospital_name: basic_info.lab_or_hospital_name
  report_date: basic_info.reports_date
  bill_date: basic_info.bill_date
  age: basic_info.age
  gender: basic_info.gender
  tests_array: report_details
  report_name: basic_info.report_name
  page_number: basic_info.page_no
```

Folder: database

File: connection.py

Path: database\connection.py

```
import os
import pymysql.cursors
from dotenv import load_dotenv

# Load variables from local .env file (safely excluded from Git)
load_dotenv()

DB_CONFIG = {
    'host': os.environ.get('DB_HOST', 'localhost'),
    'user': os.environ.get('DB_USER', 'root'),
    'password': os.environ.get('DB_PASSWORD'),
    'database': os.environ.get('DB_NAME', 'medical_data_standardisation'),
    'charset': 'utf8mb4',
    'cursorclass': pymysql.cursors.DictCursor
}

def get_db_connection():
    """Establish and return a connection to the MySQL database."""
    return pymysql.connect(**DB_CONFIG)
```

File: generate_ddl.py

Path: database\generate_ddl.py

```
import csv
import os

TYPE_MAP = {
    "STRING": "VARCHAR(128)",
    "FLOAT64": "DOUBLE",
    "TIMESTAMP": "TIMESTAMP"
}

LONG_TEXT_FIELDS = {
    "diagnosis", "brief_history", "general_examinations", "recommendations",
    "hospital_address", "post_discharge_advice", "medicine_injections_investigation",
    "metadetails", "other_med_inj_investigations", "course_during_hospitalisation",
    "course_during_hospitalization", "result_text_original", "range_text_original",
    "error_message", "raw_payload",
    "result_text", "result", "range_text", "range", "test_name_original", "test_name",
    "report_details_range", "report_details_result", "report_details_test_name"
}

def build_ddl(csv_path, out_path):
    rows = list(csv.DictReader(open(csv_path, encoding='utf-8')))

    cols = []
    for r in rows:
        col_name = r['column_name']
        data_type = r['data_type']

        # We handle 'id' separately to make it primary key
        if col_name == 'id':
            cols.append(f"    `id` VARCHAR(64) PRIMARY KEY")
            continue

        sql_type = TYPE_MAP.get(data_type, "VARCHAR(128)")

        # Override long text fields to TEXT
        if col_name in LONG_TEXT_FIELDS:
            sql_type = "TEXT"

        # Override processed_at and ingested_at defaults
        if col_name == 'processed_at':
            cols.append(f"    `processed_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP")
        elif col_name == 'ingested_at':
            cols.append(f"    `ingested_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP")
        else:
            cols.append(f"    `{col_name}` {sql_type} NULL")
```

```

# Add operational columns (except processed_at which is already in CSV)
cols.append("    processing_status VARCHAR(16) DEFAULT 'processed'")
cols.append("    duplicate_flag BOOLEAN DEFAULT FALSE")
cols.append("    error_message TEXT NULL")

# Add Unique Index
cols.append("    UNIQUE KEY uq_dedupe (claim_no(100), document_id(100),
record_type(50), test_name_original(100), medicine(100), id(64))")

ddl = "CREATE DATABASE IF NOT EXISTS medical_data_standardisation CHARACTER SET
utf8mb4;\n"
ddl += "USE medical_data_standardisation;\n\n"

ddl += "DROP TABLE IF EXISTS standardized_records;\n"
ddl += "CREATE TABLE standardized_records (\n"
ddl += ",\n".join(cols)
ddl += "\n) ENGINE=InnoDB;\n\n"

ddl += "DROP TABLE IF EXISTS ingestion_errors;\n"
ddl += """CREATE TABLE ingestion_errors (
error_id      BIGINT AUTO_INCREMENT PRIMARY KEY,
file_path     VARCHAR(512),
document_id   VARCHAR(64) NULL,
stage         VARCHAR(32),      -- ingest | standardize | validate | load
error_reason  TEXT,
raw_payload   LONGTEXT,
occurred_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;\n\n"""

ddl += "DROP TABLE IF EXISTS pipeline_runs;\n"
ddl += """CREATE TABLE pipeline_runs (
run_id        BIGINT AUTO_INCREMENT PRIMARY KEY,
started_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
finished_at   TIMESTAMP NULL,
files_seen    INT DEFAULT 0,
files_processed INT DEFAULT 0,
files_failed  INT DEFAULT 0,
records_flagged INT DEFAULT 0,
duplicates_skipped INT DEFAULT 0
) ENGINE=InnoDB;\n\n"""

with open(out_path, "w", encoding='utf-8') as f:
    f.write(ddl)
print(f"Generated DDL at {out_path}")

if __name__ == "__main__":
    csv_file = os.path.join(os.path.dirname(__file__), "Ourput-table-ideal-schema.csv")
    sql_file = os.path.join(os.path.dirname(__file__), "schema.sql")
    build_ddl(csv_file, sql_file)

```

File: Ourput-table-ideal-schema.csv

Path: database\Ourput-table-ideal-schema.csv

```

column_name,data_type,is_nullable
id,STRING,YES
document_id,STRING,YES
record_type,STRING,YES
file_gcs_path,STRING,YES
trace_id,STRING,YES
correlation_id,STRING,YES
source_system,STRING,YES
claim_no,STRING,YES
nt_code,STRING,YES
consumer_client_id,STRING,YES
destination_identifier,STRING,YES
patient_name,STRING,YES
age,STRING,YES
gender,STRING,YES
uhid,STRING,YES
hospital_name,STRING,YES
doctor_name,STRING,YES
bill_date,STRING,YES
reports_date,STRING,YES
test_name_canonical,STRING,YES
test_name_original,STRING,YES
result_value,FLOAT64,YES
result_text,STRING,YES
unit_canonical,STRING,YES
unit_original,STRING,YES
range_low,FLOAT64,YES
range_high,FLOAT64,YES
range_text,STRING,YES
test_analytics,STRING,YES
normalization_method,STRING,YES
normalization_confidence,FLOAT64,YES
admission_date,STRING,YES
discharge_date,STRING,YES
diagnosis,STRING,YES
brief_history,STRING,YES
general_examinations,STRING,YES
recommendations,STRING,YES
hospital_address,STRING,YES
ward,STRING,YES
post_discharge_advice,STRING,YES
medicine,STRING,YES
dose,STRING,YES
frequency,STRING,YES
medicine_type,STRING,YES
processed_at,TIMESTAMP,YES
ingested_at,TIMESTAMP,YES

```

```
basic_info_age,STRING,YES
basic_info_bill_date,STRING,YES
medicine_injections_investigation,STRING,YES
discharge_medications_dose,STRING,YES
metadetails,STRING,YES
discharge_medications_frequency,STRING,YES
discharge_medications_medicine,STRING,YES
lab_or_hospital_name,STRING,YES
report_details_page_no,STRING,YES
report_details_range,STRING,YES
report_details_result,STRING,YES
report_details_test_analytics,STRING,YES
report_details_test_name,STRING,YES
report_details_unit,STRING,YES
age_text,STRING,YES
other_med_inj_investigations,STRING,YES
report_date,STRING,YES
course_during_hospitalisation,STRING,YES
page_number,STRING,YES
range_text_original,STRING,YES
medication_dose,STRING,YES
result_text_original,STRING,YES
medication_frequency,STRING,YES
course_during_hospitalization,STRING,YES
medication_name,STRING,YES
page_no,STRING,YES
test_name,STRING,YES
medication_medicine,STRING,YES
result,STRING,YES
unit,STRING,YES
age_years,STRING,YES
range,STRING,YES
```

File: schema.sql

Path: database\schema.sql

```
CREATE DATABASE IF NOT EXISTS medical_data_standardisation CHARACTER SET utf8mb4;
USE medical_data_standardisation;
```

```
DROP TABLE IF EXISTS standardized_records;
CREATE TABLE standardized_records (
  `id` VARCHAR(64) PRIMARY KEY,
  `document_id` VARCHAR(128) NULL,
  `record_type` VARCHAR(128) NULL,
  `file_gcs_path` VARCHAR(128) NULL,
  `trace_id` VARCHAR(128) NULL,
  `correlation_id` VARCHAR(128) NULL,
  `source_system` VARCHAR(128) NULL,
  `claim_no` VARCHAR(128) NULL,
  `nt_code` VARCHAR(128) NULL,
  `consumer_client_id` VARCHAR(128) NULL,
  `destination_identifier` VARCHAR(128) NULL,
  `patient_name` VARCHAR(128) NULL,
  `age` VARCHAR(128) NULL,
  `gender` VARCHAR(128) NULL,
  `uhid` VARCHAR(128) NULL,
  `hospital_name` VARCHAR(128) NULL,
  `doctor_name` VARCHAR(128) NULL,
  `bill_date` VARCHAR(128) NULL,
  `reports_date` VARCHAR(128) NULL,
  `test_name_canonical` VARCHAR(128) NULL,
  `test_name_original` TEXT NULL,
  `result_value` DOUBLE NULL,
  `result_text` TEXT NULL,
  `unit_canonical` VARCHAR(128) NULL,
  `unit_original` VARCHAR(128) NULL,
  `range_low` DOUBLE NULL,
  `range_high` DOUBLE NULL,
  `range_text` TEXT NULL,
  `test_analytics` VARCHAR(128) NULL,
  `normalization_method` VARCHAR(128) NULL,
  `normalization_confidence` DOUBLE NULL,
  `admission_date` VARCHAR(128) NULL,
  `discharge_date` VARCHAR(128) NULL,
  `diagnosis` TEXT NULL,
  `brief_history` TEXT NULL,
  `general_examinations` TEXT NULL,
  `recommendations` TEXT NULL,
  `hospital_address` TEXT NULL,
  `ward` VARCHAR(128) NULL,
  `post_discharge_advice` TEXT NULL,
  `medicine` VARCHAR(128) NULL,
  `dose` VARCHAR(128) NULL,
```



```

`frequency` VARCHAR(128) NULL,
`medicine_type` VARCHAR(128) NULL,
`processed_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
`ingested_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
`basic_info_age` VARCHAR(128) NULL,
`basic_info_bill_date` VARCHAR(128) NULL,
`medicine_injections_investigation` TEXT NULL,
`discharge_medications_dose` VARCHAR(128) NULL,
`metadetails` TEXT NULL,
`discharge_medications_frequency` VARCHAR(128) NULL,
`discharge_medications_medicine` VARCHAR(128) NULL,
`lab_or_hospital_name` VARCHAR(128) NULL,
`report_details_page_no` VARCHAR(128) NULL,
`report_details_range` TEXT NULL,
`report_details_result` TEXT NULL,
`report_details_test_analytics` VARCHAR(128) NULL,
`report_details_test_name` TEXT NULL,
`report_details_unit` VARCHAR(128) NULL,
`age_text` VARCHAR(128) NULL,
`other_med_inj_investigations` TEXT NULL,
`report_date` VARCHAR(128) NULL,
`course_during_hospitalisation` TEXT NULL,
`page_number` VARCHAR(128) NULL,
`range_text_original` TEXT NULL,
`medication_dose` VARCHAR(128) NULL,
`result_text_original` TEXT NULL,
`medication_frequency` VARCHAR(128) NULL,
`course_during_hospitalization` TEXT NULL,
`medication_name` VARCHAR(128) NULL,
`page_no` VARCHAR(128) NULL,
`test_name` TEXT NULL,
`medication_medicine` VARCHAR(128) NULL,
`result` TEXT NULL,
`unit` VARCHAR(128) NULL,
`age_years` VARCHAR(128) NULL,
`range` TEXT NULL,
processing_status VARCHAR(16) DEFAULT 'processed',
duplicate_flag BOOLEAN DEFAULT FALSE,
error_message TEXT NULL,
UNIQUE KEY uq_dedupe (claim_no(100), document_id(100), record_type(50),
test_name_original(100), medicine(100), id(64))
) ENGINE=InnoDB;

DROP TABLE IF EXISTS ingestion_errors;
CREATE TABLE ingestion_errors (
    error_id      BIGINT AUTO_INCREMENT PRIMARY KEY,
    file_path     VARCHAR(512),
    document_id   VARCHAR(64) NULL,
    stage         VARCHAR(32),      -- ingest | standardize | validate | load
    error_reason  TEXT,
    raw_payload   LONGTEXT,
    occurred_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

```

```
DROP TABLE IF EXISTS pipeline_runs;  
CREATE TABLE pipeline_runs (  
    run_id          BIGINT AUTO_INCREMENT PRIMARY KEY,  
    started_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    finished_at     TIMESTAMP NULL,  
    files_seen      INT DEFAULT 0,  
    files_processed INT DEFAULT 0,  
    files_failed    INT DEFAULT 0,  
    records_flagged INT DEFAULT 0,  
    duplicates_skipped INT DEFAULT 0  
) ENGINE=InnoDB;
```

Folder: src

File: db_loader.py

Path: src\db_loader.py

```
import sys
import pymysql
from database.connection import get_db_connection

def insert_standardized_records(records):
    """
    Inserts or updates standardized records in MySQL.
    """
    if not records:
        return 0, 0

    connection = get_db_connection()
    inserted_count = 0
    duplicate_count = 0

    try:
        # Clean records to leverage MySQL defaults for timestamps, and ensure operational
        # keys exist
        for record in records:
            record.pop('processed_at', None)
            record.pop('ingested_at', None)
            record['duplicate_flag'] = False
            record.setdefault('error_message', None)

        with connection.cursor() as cursor:
            # We dynamically build the SQL query based on the fields of the first record
            # to accommodate all 78 schema columns.
            sample_record = records[0]
            columns = list(sample_record.keys())

            escaped_cols = [f"`{col}`" for col in columns]
            placeholders = [f"%({col})s" for col in columns]

            # Build ON DUPLICATE KEY UPDATE clause
            update_clauses = []
            for col in columns:
                if col == 'id':
                    continue
                # Mark as duplicate if a conflict occurs
                if col == 'duplicate_flag':
                    update_clauses.append("`duplicate_flag` = TRUE")
                else:
                    update_clauses.append(f"`{col}` = VALUES(`{col}`)")

            sql = f"""
            INSERT INTO standardized_records ({', '.join(escaped_cols)})
            VALUES ({', '.join(placeholders)})
            """
```

```

        ON DUPLICATE KEY UPDATE {'', '.join(update_clauses)}
    """

    # Execute records one by one to count updates vs inserts
    for record in records:
        try:
            cursor.execute(sql, record)
            # MySQL rowcount: 1 = inserted, 2 = updated (changed), 0 = updated (no
change)
            if cursor.rowcount == 1:
                inserted_count += 1
            else:
                duplicate_count += 1
        except Exception as e:
            print(f"Error inserting record ID {record.get('id')}: {e}",
file=sys.stderr)
            # We write to ingestion errors
            log_db_error('standardized_records', record.get('document_id'), 'load',
f"RecordLoadError: {str(e)}", str(record))

        connection.commit()
    finally:
        connection.close()

    return inserted_count, duplicate_count

def log_db_error(file_path, document_id, stage, error_reason, raw_payload):
    """Insert error log into ingestion_errors table."""
    connection = get_db_connection()
    try:
        with connection.cursor() as cursor:
            sql = """
                INSERT INTO ingestion_errors (file_path, document_id, stage, error_reason,
raw_payload)
                VALUES (%s, %s, %s, %s, %s)
            """
            cursor.execute(sql, (file_path, document_id, stage, error_reason, raw_payload))
            connection.commit()
    except Exception as e:
        print(f"Failed to log error to DB: {e}", file=sys.stderr)
    finally:
        connection.close()

def start_pipeline_run():
    """Log the start of a pipeline run in pipeline_runs."""
    connection = get_db_connection()
    run_id = None
    try:
        with connection.cursor() as cursor:
            sql = "INSERT INTO pipeline_runs (started_at) VALUES (CURRENT_TIMESTAMP)"
            cursor.execute(sql)
            run_id = cursor.lastrowid
            connection.commit()
    finally:

```

```

        connection.close()
    return run_id

def finish_pipeline_run(run_id, files_seen, files_processed, files_failed, records_flagged,
duplicates_skipped):
    """Update pipeline_runs record upon completion."""
    connection = get_db_connection()
    try:
        with connection.cursor() as cursor:
            sql = """
                UPDATE pipeline_runs
                SET finished_at = CURRENT_TIMESTAMP,
                    files_seen = %s,
                    files_processed = %s,
                    files_failed = %s,
                    records_flagged = %s,
                    duplicates_skipped = %s
                WHERE run_id = %s
            """
            cursor.execute(sql, (files_seen, files_processed, files_failed,
records_flagged, duplicates_skipped, run_id))
            connection.commit()
        finally:
            connection.close()

def reset_database():
    """Wipes the database tables to start with a fresh state, preserving run history
telemetry."""
    connection = get_db_connection()
    try:
        with connection.cursor() as cursor:
            cursor.execute("SET FOREIGN_KEY_CHECKS = 0;")
            cursor.execute("TRUNCATE TABLE standardized_records;")
            cursor.execute("TRUNCATE TABLE ingestion_errors;")
            # We do not truncate pipeline_runs to preserve execution history in the
dashboard
            cursor.execute("SET FOREIGN_KEY_CHECKS = 1;")
            connection.commit()
            print("Database clinical tables truncated successfully. Running fresh load...")
        finally:
            connection.close()

```

File: ingest.py

Path: src\ingest.py

```
import os
import json
from src.utils import load_yaml

def lookup_path(data_dict, path):
    """Retrieve values from a dictionary using dot-separated paths (e.g.,
    'basic_info.patient_name')."""
    if not path or not isinstance(data_dict, dict):
        return None
    keys = path.split('.')
    curr = data_dict
    for key in keys:
        if isinstance(curr, dict) and key in curr:
            curr = curr[key]
        else:
            return None
    return curr

def resolve_field(data_dict, field_name, doc_config, alias_config):
    """
    Resolve a field value from document data using:
    1. Document-type specific mappings.
    2. Fallback field aliases list.
    """
    # 1. Try document-type specific mapping
    doc_fields = doc_config.get('fields', {})
    if field_name in doc_fields:
        mapped_path = doc_fields[field_name]
        val = lookup_path(data_dict, mapped_path)
        if val is not None and val != "":
            return val

    # 2. Try generic field aliases
    aliases = alias_config.get(field_name, [])
    for alias in aliases:
        val = lookup_path(data_dict, alias)
        if val is not None and val != "":
            return val

    return None

def ingest_file(file_path, doc_configs, alias_config):
    """
    Ingest a single JSON file.
    Returns: (list of parsed records, error_dict or None)
    """
    file_name = os.path.basename(file_path)
```

```

try:
    with open(file_path, 'r', encoding='utf-8') as f:
        payload = json.load(f)
except json.JSONDecodeError as jde:
    # Capture raw payload for audit trail (FR-4.3)
    try:
        with open(file_path, 'r', encoding='utf-8') as f:
            raw_text = f.read()
    except Exception:
        raw_text = "Could not read file text"

    error_info = {
        'file_path': file_path,
        'document_id': None,
        'stage': 'ingest',
        'error_reason': f"JSONDecodeError: {str(jde)}",
        'raw_payload': raw_text
    }
    return [], error_info
except Exception as e:
    error_info = {
        'file_path': file_path,
        'document_id': None,
        'stage': 'ingest',
        'error_reason': f"FileError: {str(e)}",
        'raw_payload': ""
    }
    return [], error_info

parsed_records = []

# Check trace details
trace_id = payload.get('traceId')
data_node = payload.get('data', {})
correlation_id = data_node.get('correlationId')
document_id = data_node.get('documentId')
meta_details = data_node.get('metaDetails', [])

# Parse metaDetails array of key/value pairs
meta_map = {}
if isinstance(meta_details, list):
    for item in meta_details:
        if isinstance(item, dict) and 'key' in item and 'value' in item:
            meta_map[item['key']] = item['value']

# Extract meta fields
claim_no = meta_map.get('claim_no')
nt_code = meta_map.get('nt_code')
source_system = meta_map.get('source_system')
consumer_client_id = meta_map.get('ConsumerClientId')
destination_identifier = meta_map.get('DestinationIdentifier')

response_details = data_node.get('responseDetails', [])
if not isinstance(response_details, list):

```

```
response_details = []

for detail in response_details:
    if not isinstance(detail, dict):
        continue

    classifier = detail.get('classifier')
    detail_data = detail.get('data', {})

    # Select matching document configuration
    doc_config = doc_configs.get(classifier, {})

    # Resolve basic demographic and clinical header fields
    patient_name = resolve_field(detail_data, 'patient_name', doc_config, alias_config)
    age = resolve_field(detail_data, 'age', doc_config, alias_config)
    gender = resolve_field(detail_data, 'gender', doc_config, alias_config)
    hospital_name = resolve_field(detail_data, 'hospital_name', doc_config,
alias_config)
    doctor_name = resolve_field(detail_data, 'doctor_name', doc_config, alias_config)

    # Build common record header
    record_header = {
        'document_id': document_id,
        'record_type': classifier,
        'file_gcs_path': f"gs://medical-records/{file_name}",
        'trace_id': trace_id,
        'correlation_id': correlation_id,
        'source_system': source_system,
        'claim_no': claim_no,
        'nt_code': nt_code,
        'consumer_client_id': consumer_client_id,
        'destination_identifier': destination_identifier,
        'patient_name': patient_name,
        'age': age,
        'gender': gender,
        'hospital_name': hospital_name,
        'doctor_name': doctor_name,
        'raw_detail_data': detail_data, # keep for extraction phase
        'raw_meta_details': json.dumps(meta_details) if meta_details else None
    }

    parsed_records.append(record_header)

return parsed_records, None
```


File: pipeline.py

Path: src\pipeline.py

```
import os
import sys
import glob
import argparse
from src.utils import load_yaml
from src.ingest import ingest_file
from src.standardize import standardize_document
from src.db_loader import (
    start_pipeline_run,
    finish_pipeline_run,
    insert_standardized_records,
    log_db_error,
    reset_database
)

def run_pipeline(data_dir=None, reset=False):
    # Resolve project paths
    project_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

    if not data_dir:
        data_dir = os.path.join(project_root, "sample-data")

    config_dir = os.path.join(project_root, "config")

    print("Initializing Veritas Claims Pipeline Run...")

    if reset:
        reset_database()

    # 1. Start pipeline run logging in the database
    run_id = start_pipeline_run()
    print(f"Pipeline Run Started. Run ID: {run_id}")

    # 2. Load configurations
    doc_configs = {
        'discharge_summary': load_yaml(os.path.join(config_dir, "document_types",
            "discharge_summary.yaml")),
        'lab_report': load_yaml(os.path.join(config_dir, "document_types",
            "lab_report.yaml"))
    }
    alias_config = load_yaml(os.path.join(config_dir, "field_aliases.yaml"))
    test_dict = load_yaml(os.path.join(config_dir, "test_name_dictionary.yaml"))
    unit_conv = load_yaml(os.path.join(config_dir, "unit_conversion.yaml"))
    ref_ranges = load_yaml(os.path.join(config_dir, "reference_ranges.yaml"))
    med_dict = load_yaml(os.path.join(config_dir, "medicine_dictionary.yaml"))
    analytics_rules = load_yaml(os.path.join(config_dir, "analytics_rules.yaml"))
```

```
# Check data directory
if not os.path.exists(data_dir):
    print(f"Error: Data directory not found at {data_dir}", file=sys.stderr)
    finish_pipeline_run(run_id, 0, 0, 1, 0, 0)
    return

# Search for all JSON files in the sample-data directory
search_path = os.path.join(data_dir, "*.json")
json_files = glob.glob(search_path)

files_seen = len(json_files)
files_processed = 0
files_failed = 0
records_flagged = 0
duplicates_skipped = 0
total_rows_inserted = 0

print(f"Scanning directory: {data_dir}. Found {files_seen} files.")

for fpath in json_files:
    fname = os.path.basename(fpath)
    print(f"Processing: {fname}...", end="")

    # Ingest file
    records, error_info = ingest_file(fpath, doc_configs, alias_config)

    if error_info:
        print(" FAILED (Ingestion Error)")
        # Log failure to database (FR-4.2)
        log_db_error(
            file_path=fpath,
            document_id=error_info.get('document_id'),
            stage=error_info.get('stage'),
            error_reason=error_info.get('error_reason'),
            raw_payload=error_info.get('raw_payload')
        )
        files_failed += 1
        continue

    # Standardise and flatten records
    flattened_rows = []
    for record_header in records:
        # Standardise document
        rows = standardize_document(
            record_header=record_header,
            unit_conv_dict=unit_conv,
            test_dict=test_dict,
            ref_ranges=ref_ranges,
            med_dict=med_dict,
            analytics_rules=analytics_rules
        )
        flattened_rows.extend(rows)

    # Apply validation checks and determine operational status flags
```

```

for row in flattened_rows:
    # Outliers or Invalid results are flagged (FR-3.4 / FR-5.1)
    is_anomaly = row['test_analytics'] in ['Below Range', 'Above Range', 'Outlier',
'Invalid']
    if is_anomaly:
        row['processing_status'] = 'flagged'
        records_flagged += 1
    else:
        row['processing_status'] = 'processed'

    # If the result value is null and result text is not empty for lab reports,
    # this represents a parsing warning (e.g. Range-as-Result or non-numeric
result)
    if row['record_type'] == 'lab_report' and row['result_value'] is None and
row['result_text']:
        if row['test_analytics'] == 'Invalid':
            row['error_message'] = "Value could not be parsed numerically (warning:
text or range-as-result)."
            row['processing_status'] = 'flagged'
            # Count as flagged if not already counted
            if not is_anomaly:
                records_flagged += 1

    # Load into database
    if flattened_rows:
        inserted, duplicates = insert_standardized_records(flattened_rows)
        total_rows_inserted += inserted
        duplicates_skipped += duplicates
        print(f" SUCCESS. Flattened into {len(flattened_rows)} rows (Inserted:
{inserted}, Duplicates: {duplicates})")
    else:
        print(" SUCCESS (No records found)")

    files_processed += 1

# 3. Finalize run logging in the database
finish_pipeline_run(
    run_id=run_id,
    files_seen=files_seen,
    files_processed=files_processed,
    files_failed=files_failed,
    records_flagged=records_flagged,
    duplicates_skipped=duplicates_skipped
)

print("\n" + "="*40)
print("Veritas Claims Pipeline Run Completed!")
print(f"Total Files Scanned:  {files_seen}")
print(f"Files Processed:      {files_processed}")
print(f"Files Failed:          {files_failed}")
print(f"Total Rows Inserted:    {total_rows_inserted}")
print(f"Duplicates Flagged:     {duplicates_skipped}")
print(f"Records Flagged (Anomalies): {records_flagged}")
print("="*40)

```

```
if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Veritas Claims Ingestion &
Standardisation Pipeline")
    parser.add_argument("--data-dir", help="Path to sample data folder containing JSON
files")
    parser.add_argument("--reset", action="store_true", help="Clear all database tables
before processing")
    args = parser.parse_args()

    run_pipeline(args.data_dir, reset=args.reset)
```

File: standardize.py

Path: src\standardize.py

```
import re
from datetime import datetime
from src.utils import string_similarity, make_id
from src.validate import parse_reference_range, classify_result

def normalize_test_name(original_name, dictionary):
    """
    Standardises test names using alias matching and Levenshtein similarity.
    Returns: (canonical_name, normalization_method, confidence)
    """
    if not original_name:
        return None, "unmapped_fallback", 0.0

    orig_clean = original_name.strip()

    # 1. Exact or case-insensitive alias match
    for canonical, info in dictionary.items():
        aliases = info.get('aliases', [])
        # Check canonical itself
        if orig_clean.lower() == canonical.lower():
            return canonical, "exact_alias", 1.0
        # Check aliases
        for alias in aliases:
            if orig_clean.lower() == alias.lower():
                return canonical, "exact_alias", 1.0

    # 2. Fuzzy match
    best_match = None
    best_score = 0.0

    for canonical, info in dictionary.items():
        aliases = info.get('aliases', [])
        # Check canonical
        score = string_similarity(orig_clean, canonical)
        if score > best_score:
            best_score = score
            best_match = canonical
        # Check aliases
        for alias in aliases:
            score = string_similarity(orig_clean, alias)
            if score > best_score:
                best_score = score
                best_match = canonical

    if best_score >= 0.8:
        return best_match, "fuzzy_match", round(best_score, 2)
```

```

# 3. Fallback
return orig_clean.upper(), "unmapped_fallback", 0.0

def extract_numeric(val_str):
    """
    Extract a floating point value from the result text.
    Returns None if the result contains a range (e.g. '4.50-5.50') or no numbers.
    """
    if not val_str:
        return None
    val_clean = val_str.replace(',', '').strip()

    # Check for range patterns like X-Y or X - Y
    if re.search(r'\d+\s*-\s*\d+', val_clean):
        return None

    # Find first decimal or integer number
    match = re.search(r'[-+]?[0-9]*\.?[0-9]+', val_clean)
    if match:
        try:
            return float(match.group())
        except ValueError:
            return None
    return None

def normalize_gender(gender_str):
    """Standardise gender representations to Male/Female/Other."""
    if not gender_str:
        return None
    g = gender_str.strip().lower()
    if "redacted" in g:
        return gender_str.strip() # Preserve REDACTED tags
    if g.startswith('m'):
        return "Male"
    if g.startswith('f'):
        return "Female"
    return "Other"

def normalize_age(age_str):
    """Standardise age representation."""
    if not age_str:
        return None
    return age_str.strip()

def normalize_date(date_str):
    """Convert input date strings to YYYY-MM-DD standard."""
    if not date_str:
        return None
    date_str = date_str.strip()

    formats = [
        ("%d-%m-%Y", False),
        ("%d-%b-%Y", False),
        ("%d/%b/%Y", False),
    ]

```

```

        ("%d/%m/%Y", False),
        ("%Y-%m-%d", True),
    ]
    for fmt, is_ok in formats:
        try:
            parsed = datetime.strptime(date_str, fmt)
            return parsed.strftime("%Y-%m-%d")
        except ValueError:
            continue
    return date_str

def normalize_unit(unit_str, conversion_dict):
    """Standardise original units to canonical units and return conversion factor."""
    if not unit_str:
        return "", 1.0
    u_clean = unit_str.strip()
    if u_clean in conversion_dict:
        info = conversion_dict[u_clean]
        if isinstance(info, dict):
            return info.get('canonical', u_clean), float(info.get('factor', 1.0))
    return u_clean, 1.0

def map_medicine(med_name, med_dict):
    """Map medication to standard canonical name and dosage type."""
    if not med_name:
        return None, None
    m_clean = med_name.strip()
    m_clean_lower = m_clean.lower()

    # Match in dictionary (case-insensitive alias lookup)
    for key, info in med_dict.items():
        aliases = [a.lower() for a in info.get('aliases', [])]
        canonical = info.get('canonical', key)
        if m_clean_lower == key.lower() or m_clean_lower == canonical.lower() or
        m_clean_lower in aliases:
            return canonical, info.get('type', 'Unknown')

    # Fallback to name-based inference if not found
    m_lower = m_clean.lower()
    inferred_type = "Unknown"
    if "tab" in m_lower:
        inferred_type = "Tablet"
    elif "inj" in m_lower:
        inferred_type = "Injection"
    elif "cap" in m_lower:
        inferred_type = "Capsule"
    elif "syp" in m_lower or "syrup" in m_lower:
        inferred_type = "Syrup"
    elif "powder" in m_lower:
        inferred_type = "Powder"

    return m_clean, inferred_type

def standardize_document(record_header, unit_conv_dict, test_dict, ref_ranges, med_dict,

```

```

analytics_rules=None):
    """
    Standardises and flattens a single document into 1 or more rows matching the 78
columns.
    """
    raw_data = record_header['raw_detail_data']
    rec_type = record_header['record_type']

    # Pre-calculate common standardized metadata
    std_patient_name = record_header['patient_name']
    std_age = normalize_age(record_header['age'])
    std_gender = normalize_gender(record_header['gender'])

    # Build standard row template with all 78 schema columns set to None
    base_row = {
        'id': None, 'document_id': record_header['document_id'], 'record_type': rec_type,
        'file_gcs_path': record_header['file_gcs_path'], 'trace_id':
record_header['trace_id'],
        'correlation_id': record_header['correlation_id'], 'source_system':
record_header['source_system'],
        'claim_no': record_header['claim_no'], 'nt_code': record_header['nt_code'],
        'consumer_client_id': record_header['consumer_client_id'],
'destination_identifier': record_header['destination_identifier'],
        'patient_name': std_patient_name, 'age': std_age, 'gender': std_gender, 'uhid':
raw_data.get('uhid'),
        'hospital_name': record_header['hospital_name'], 'doctor_name':
record_header['doctor_name'],
        'bill_date': None, 'reports_date': None,
        'test_name_canonical': None, 'test_name_original': None,
        'result_value': None, 'result_text': None, 'unit_canonical': None, 'unit_original':
None,
        'range_low': None, 'range_high': None, 'range_text': None, 'test_analytics': None,
        'normalization_method': None, 'normalization_confidence': None,
        'admission_date': None, 'discharge_date': None,
        'diagnosis': None, 'brief_history': None, 'general_examinations': None,
'recommendations': None,
        'hospital_address': raw_data.get('hospitalAddress'), 'ward': raw_data.get('ward'),
        'post_discharge_advice': raw_data.get('postDischargeAdvice'),
        'medicine': None, 'dose': None, 'frequency': None, 'medicine_type': None,
        'processed_at': None, 'ingested_at': None,
        # Legacy/Duplicate columns
        'basic_info_age': raw_data.get('age') if rec_type == 'discharge_summary' else
raw_data.get('basic_info', {}).get('age'),
        'basic_info_bill_date': raw_data.get('basic_info', {}).get('bill_date'),
        'medicine_injections_investigation':
str(raw_data.get('medicineInjectionsInvestigation')) if
raw_data.get('medicineInjectionsInvestigation') else None,
        'discharge_medications_dose': None,
        'metadetails': record_header['raw_meta_details'],
        'discharge_medications_frequency': None,
        'discharge_medications_medicine': None,
        'lab_or_hospital_name': raw_data.get('basic_info', {}).get('lab_or_hospital_name'),
        'report_details_page_no': None, 'report_details_range': None,
        'report_details_result': None,

```



```

        'report_details_test_analytics': None, 'report_details_test_name': None,
        'report_details_unit': None,
        'age_text': raw_data.get('basic_info', {}).get('age'),
        'other_med_inj_investigations':
str(raw_data.get('medicineInjectionsInvestigation')) if
raw_data.get('medicineInjectionsInvestigation') else None,
        'report_date': raw_data.get('basic_info', {}).get('reports_date'),
        'course_during_hospitalisation': str(raw_data.get('courseDuringHospitalisation'))
if raw_data.get('courseDuringHospitalisation') else None,
        'page_number': None, 'range_text_original': None, 'medication_dose': None,
        'result_text_original': None,
        'medication_frequency': None, 'course_during_hospitalization':
str(raw_data.get('courseDuringHospitalisation')) if
raw_data.get('courseDuringHospitalisation') else None,
        'medication_name': None, 'page_no': None, 'test_name': None, 'medication_medicine':
None,
        'result': None, 'unit': None,
        'age_years': raw_data.get('basic_info', {}).get('age'),
        'range': None
    }

    rows = []

    if rec_type == 'lab_report':
        # Standardise date fields
        base_row['bill_date'] = normalize_date(raw_data.get('basic_info',
{}).get('bill_date'))
        base_row['reports_date'] = normalize_date(raw_data.get('basic_info',
{}).get('reports_date'))
        base_row['uhid'] = raw_data.get('basic_info', {}).get('uhid')

        report_details = raw_data.get('report_details', [])
        if not report_details:
            # Fallback to single empty test row if array is empty
            row = base_row.copy()
            row['id'] = make_id(row['claim_no'], row['document_id'], row['record_type'],
"empty_report")
            rows.append(row)
        else:
            for idx, test in enumerate(report_details):
                row = base_row.copy()
                orig_name = test.get('test_name')

                # Deterministic ID includes claim_no, document_id, record_type, and
discriminator (original test name + index)
                row_id = make_id(row['claim_no'], row['document_id'], row['record_type'],
f"{orig_name or 'none'}|{idx}")
                row['id'] = row_id

                orig_result = test.get('result')
                orig_unit = test.get('unit')
                orig_range = test.get('range')
                page_no = test.get('page_no')
                orig_analytics = test.get('test_analytics')

```

```

test_dict)

    # Test name normalisation
    canon_name, norm_method, norm_conf = normalize_test_name(orig_name,

    row['test_name_canonical'] = canon_name
    row['test_name_original'] = orig_name
    row['normalization_method'] = norm_method
    row['normalization_confidence'] = norm_conf

    # Unit normalisation & conversion factor scaling
    row['unit_original'] = orig_unit
    row['unit'] = orig_unit
    canon_unit, factor = normalize_unit(orig_unit, unit_conv_dict)
    row['unit_canonical'] = canon_unit

    # Result parsing
    row['result_text'] = orig_result
    row['result'] = orig_result
    row['result_text_original'] = orig_result
    res_val = extract_numeric(orig_result)
    if res_val is not None:
        res_val = float(res_val) * float(factor)
    row['result_value'] = res_val

    # Reference range parsing
    row['range_text'] = orig_range
    row['range'] = orig_range
    row['range_text_original'] = orig_range
    low, high = parse_reference_range(orig_range)
    if low is not None:
        low = float(low) * float(factor)
    if high is not None:
        high = float(high) * float(factor)
    row['range_low'] = low
    row['range_high'] = high

    # Outlier / Validation Classification
    low_calc = None
    high_calc = None
    outlier_low = None
    outlier_high = None

    # Check if test has configuration thresholds
    has_config = canon_name in ref_ranges
    if has_config:
        outlier_low = ref_ranges[canon_name].get('outlier_low')
        outlier_high = ref_ranges[canon_name].get('outlier_high')

    # Determine which range boundaries to use
    # If document provides a range (Rule 2 & Rule 4), use the parsed document
    range (low, high)
    if low is not None or high is not None:
        low_calc = low
        high_calc = high

```

```

# Else, if present in reference_ranges.yaml (Rule 3), use configuration
range
    elif has_config and (ref_ranges[canon_name].get('low') is not None or
ref_ranges[canon_name].get('high') is not None):
        low_calc = ref_ranges[canon_name].get('low')
        high_calc = ref_ranges[canon_name].get('high')

# Classify based on resolved range boundaries
if low_calc is not None or high_calc is not None:
    row['test_analytics'] = classify_result(
        row['result_value'], low_calc, high_calc, outlier_low,
outlier_high,
        raw_text=orig_result, rules=analytics_rules, canon_name=canon_name,
ref_ranges=ref_ranges
    )
else:
    # No range is available in either the document or reference_ranges.yaml
    # Rule 1: Check if it matches a valid qualitative test in the global
rules
        qualitative_analytics = classify_result(
            row['result_value'], None, None, None, None,
            raw_text=orig_result, rules=analytics_rules, canon_name=canon_name,
ref_ranges=ref_ranges
        )
        if qualitative_analytics in ["Within Range", "Above Range"]:
            row['test_analytics'] = qualitative_analytics
        else:
            # Numeric test with no range is Invalid
            row['test_analytics'] = "Invalid"

# Save raw analytics value before mapping fallbacks
row['report_details_test_analytics'] = orig_analytics

# Fallback to original classifier only if it matches one of our standard
enum flags
if row['test_analytics'] == "Invalid" and orig_analytics:
    orig_clean_flag = orig_analytics.strip().lower()
    valid_flags = {
        "within range": "Within Range",
        "above range": "Above Range",
        "below range": "Below Range",
        "outlier": "Outlier",
        "invalid": "Invalid"
    }
    if orig_clean_flag in valid_flags:
        row['test_analytics'] = valid_flags[orig_clean_flag]

# Page numbers & duplicates fields
row['page_no'] = str(page_no) if page_no is not None else None
row['page_number'] = str(page_no) if page_no is not None else None

row['report_details_page_no'] = str(page_no) if page_no is not None else
None
row['report_details_range'] = orig_range

```

```

        row['report_details_result'] = orig_result
        row['report_details_test_analytics'] = orig_analytics
        row['report_details_test_name'] = orig_name
        row['report_details_unit'] = orig_unit
        row['test_name'] = orig_name

        rows.append(row)

elif rec_type == 'discharge_summary':
    # Standardise date fields
    base_row['admission_date'] = normalize_date(raw_data.get('admissionDate'))
    base_row['discharge_date'] = normalize_date(raw_data.get('dischargeDate'))
    base_row['diagnosis'] = raw_data.get('diagnosis')
    base_row['brief_history'] = raw_data.get('briefHistory')
    base_row['general_examinations'] = raw_data.get('generalExaminations')
    base_row['recommendations'] = raw_data.get('recommendations')

    discharge_meds = raw_data.get('dischargeMedications', [])
    if not discharge_meds:
        # Fallback to single empty row if no medicines are present (Section 5)
        row = base_row.copy()
        row['id'] = make_id(row['claim_no'], row['document_id'], row['record_type'],
"empty_discharge")
        rows.append(row)
    else:
        for idx, med in enumerate(discharge_meds):
            row = base_row.copy()
            med_name = med.get('medicine')

            # Deterministic ID includes claim_no, document_id, record_type, and
discriminator (original medicine name + index)
            row_id = make_id(row['claim_no'], row['document_id'], row['record_type'],
f"{med_name or 'none'}|{idx}")
            row['id'] = row_id

            med_dose = med.get('dose')
            med_freq = med.get('frequency')

            # Medication map
            canon_med, med_type = map_medicine(med_name, med_dict)
            row['medicine'] = canon_med
            row['medication_medicine'] = med_name
            row['medication_name'] = med_name
            row['discharge_medications_medicine'] = med_name

            row['dose'] = med_dose
            row['medication_dose'] = med_dose
            row['discharge_medications_dose'] = med_dose

            row['frequency'] = med_freq
            row['medication_frequency'] = med_freq
            row['discharge_medications_frequency'] = med_freq

            row['medicine_type'] = med_type

```

```
        rows.append(row)

    else:
        # Generic record identification
        row = base_row.copy()
        row['id'] = make_id(row['claim_no'], row['document_id'], row['record_type'],
"generic_doc")
        rows.append(row)

    return rows
```

File: utils.py

Path: src\utils.py

```
import hashlib
import os
import yaml

def levenshtein_distance(s1, s2):
    """Calculate the Levenshtein distance between two strings."""
    if len(s1) < len(s2):
        return levenshtein_distance(s2, s1)
    if len(s2) == 0:
        return len(s1)
    previous_row = range(len(s2) + 1)
    for i, c1 in enumerate(s1):
        current_row = [i + 1]
        for j, c2 in enumerate(s2):
            insertions = previous_row[j + 1] + 1
            deletions = current_row[j] + 1
            substitutions = previous_row[j] + (c1 != c2)
            current_row.append(min(insertions, deletions, substitutions))
        previous_row = current_row
    return previous_row[-1]

def string_similarity(s1, s2):
    """Calculate the normalized similarity ratio between two strings (0.0 to 1.0)."""
    s1, s2 = s1.lower().strip(), s2.lower().strip()
    if s1 == s2:
        return 1.0
    if not s1 or not s2:
        return 0.0
    max_len = max(len(s1), len(s2))
    dist = levenshtein_distance(s1, s2)
    return 1.0 - (dist / max_len)

def make_id(claim_no, document_id, record_type, discriminator):
    """Generate a deterministic SHA256 hash to act as the primary key ID."""
    # Ensure no None values enter the hash key
    c_no = claim_no or ""
    doc_id = document_id or ""
    rec_type = record_type or ""
    disc = discriminator or ""
    raw = f"{c_no}|{doc_id}|{rec_type}|{disc}"
    return hashlib.sha256(raw.encode('utf-8')).hexdigest()

def load_yaml(file_path):
    """Utility to safely load YAML files."""
    if not os.path.exists(file_path):
        return {}
```

```
with open(file_path, 'r', encoding='utf-8') as f:  
    return yaml.safe_load(f) or {}
```

File: validate.py

Path: src\validate.py

```
import re

def parse_reference_range(range_str):
    """
    Parses original reference range strings like '4000-10000', '<50', '>60'
    into numerical (low, high) bounds.
    """
    if not range_str:
        return None, None

    range_str = range_str.replace(',', ' ').strip()

    # 1. Matches standard range format: '4000-10000' or '4.50-5.50'
    range_match = re.match(r'^([+-]?\d*\.\d+|[-+]?\d+)\s*-\s*([+-]?\d*\.\d+|[-+]?\d+)\$',
range_str)
    if range_match:
        try:
            return float(range_match.group(1)), float(range_match.group(2))
        except ValueError:
            pass

    # 2. Matches less than format: '<50' or '<= 12.5'
    lt_match = re.match(r'^<=\s*([+-]?\d*\.\d+|[-+]?\d+)|^<\s*([+-]?\d*\.\d+|[-+]?\d+)\$',
range_str)
    if lt_match:
        val = lt_match.group(1) or lt_match.group(2)
        try:
            return None, float(val)
        except ValueError:
            pass

    # 3. Matches greater than format: '>60' or '>= 135'
    gt_match = re.match(r'^>=\s*([+-]?\d*\.\d+|[-+]?\d+)|^>\s*([+-]?\d*\.\d+|[-+]?\d+)\$',
range_str)
    if gt_match:
        val = gt_match.group(1) or gt_match.group(2)
        try:
            return float(val), None
        except ValueError:
            pass

    return None, None

def classify_result(value, range_low, range_high, outlier_low=None, outlier_high=None,
raw_text=None, rules=None, canon_name=None, ref_ranges=None):
    """
    Classify a clinical result value based on reference range and outlier limits.
```



```

Supports qualitative validation using allowed text parameters.
Returns: 'Within Range' | 'Above Range' | 'Below Range' | 'Outlier' | 'Invalid'
"""
if value is None:
    if raw_text is not None:
        raw_text_str = str(raw_text).strip()

        # 1. Check reference range config if defined as qualitative
        if ref_ranges and canon_name and canon_name in ref_ranges:
            cfg = ref_ranges[canon_name]
            if cfg.get('validation_type') == 'qualitative':
                allowed = [str(a).lower() for a in cfg.get('allowed', [])]
                if raw_text_str.lower() in allowed:
                    # Treat clinical indicators (Positive, Reactive) as warning flags
                    if raw_text_str.lower() in ["positive", "reactive"]:
                        return "Above Range"
                    return "Within Range"
                else:
                    return "Invalid"

            # 2. Check global allowed values in analytics rules
            if rules:
                allowed_vals = [str(v).lower() for v in rules.get('allowed_values', [])]
                if raw_text_str.lower() in allowed_vals:
                    if raw_text_str.lower() in ["positive", "reactive"]:
                        return "Above Range"
                    return "Within Range"

            return "Invalid"

        # Check outlier limits first
        if outlier_low is not None and value <= outlier_low:
            return "Outlier"
        if outlier_high is not None and value >= outlier_high:
            return "Outlier"

        # Check normal reference ranges
        if range_low is not None and value < range_low:
            return "Below Range"
        if range_high is not None and value > range_high:
            return "Above Range"

        return "Within Range"

```

Folder: tests

File: test_standardize.py

Path: tests\test_standardize.py

```
import sys
import os

# Add project root to path
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from src.standardize import normalize_test_name, extract_numeric, normalize_date
from src.utils import make_id

# Test dictionary mapping
TEST_DICTIONARY = {
    "HAEMOGLOBIN": {
        "aliases": ["HAEMOGLOBIN", "Haemoglobin", "Hb", "haemoglobin", "aemoglobin"]
    },
    "WHITE_BLOOD_CELL_COUNT": {
        "aliases": ["TOTAL WBC COUNT", "Total WBC Count", "WBC COUNT", "tal WBC Count"]
    }
}

def test_normalize_test_name_handles_ocr_typo():
    canonical, method, score = normalize_test_name("aemoglobin", TEST_DICTIONARY)
    assert canonical == "HAEMOGLOBIN"
    assert method == "exact_alias"

    # Test fuzzy matching
    canonical, method, score = normalize_test_name("haemoglobn", TEST_DICTIONARY) # small
    typo
    assert canonical == "HAEMOGLOBIN"
    assert method == "fuzzy_match"
    assert score >= 0.8

def test_extract_numeric_strips_units():
    assert extract_numeric("120000 cells/cu.mm") == 120000.0
    assert extract_numeric("13.7 g/dl") == 13.7
    assert extract_numeric("4,290") == 4290.0
    assert extract_numeric("POSITIVE") is None
    assert extract_numeric("4.50-5.50") is None # Range, must not parse as result value

def test_normalize_date_formats():
    assert normalize_date("09-10-2025") == "2025-10-09"
    assert normalize_date("07-Oct-2025") == "2025-10-07"
    assert normalize_date("08/Oct/2025") == "2025-10-08"

def test_idempotent_id_includes_claim_no():
    id1 = make_id("CLAIM-A", "DOC001", "lab_report", "HEMOGLOBIN")
    id2 = make_id("CLAIM-B", "DOC001", "lab_report", "HEMOGLOBIN")
    assert id1 != id2 # same document_id, different claim_no -> must not collide
```

```
if __name__ == "__main__":  
    # If run directly as a script  
    print("Running standardize tests...")  
    try:  
        test_normalize_test_name_handles_ocr_typo()  
        test_extract_numeric_strips_units()  
        test_normalize_date_formats()  
        test_idempotent_id_includes_claim_no()  
        print("All standardize tests passed successfully!")  
    except AssertionError as ae:  
        print(f"Test failure: {ae}")  
        sys.exit(1)
```

File: test_validate.py

Path: tests\test_validate.py

```
import sys
import os

# Add project root to path
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from src.validate import parse_reference_range, classify_result

def test_parse_reference_range():
    # Standard range
    assert parse_reference_range("4000-10000") == (4000.0, 10000.0)
    assert parse_reference_range("4.50-5.50") == (4.5, 5.5)

    # Upper-bound only
    assert parse_reference_range("<50") == (None, 50.0)
    assert parse_reference_range("<= 12.5") == (None, 12.5)

    # Lower-bound only
    assert parse_reference_range(">60") == (60.0, None)
    assert parse_reference_range(">= 135") == (135.0, None)

    # Unparseable ranges
    assert parse_reference_range("Less than 1:80") == (None, None)
    assert parse_reference_range("Microscopy- Giemsa") == (None, None)

def test_classify_result_flags_outlier():
    # Test outlier low and high bounds
    # (value, range_low, range_high, outlier_low, outlier_high)
    assert classify_result(0.1, 12.0, 17.0, outlier_low=3.0) == "Outlier"
    assert classify_result(25.0, 12.0, 17.0, outlier_high=20.0) == "Outlier"

    # Test below and above normal ranges (but not outliers)
    assert classify_result(10.0, 12.0, 17.0, outlier_low=3.0) == "Below Range"
    assert classify_result(18.0, 12.0, 17.0, outlier_high=20.0) == "Above Range"

    # Test within normal range
    assert classify_result(15.0, 12.0, 17.0, outlier_low=3.0, outlier_high=20.0) == "Within Range"

    # Test missing results
    assert classify_result(None, 12.0, 17.0) == "Invalid"

if __name__ == "__main__":
    print("Running validate tests...")
    try:
        test_parse_reference_range()
        test_classify_result_flags_outlier()
```

```
    print("All validate tests passed successfully!")  
except AssertionError as ae:  
    print(f"Test failure: {ae}")  
    sys.exit(1)
```

Folder: ui

File: app.py

Path: ui\app.py

```
import os
import sys
from flask import Flask, render_template, request, jsonify, redirect

# Add project root to path
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from database.connection import get_db_connection
from src.pipeline import run_pipeline

app = Flask(__name__, template_folder='templates', static_folder='static')

def query_db(query, args=(), one=False):
    """Utility to execute a query and return results."""
    conn = get_db_connection()
    try:
        with conn.cursor() as cur:
            cur.execute(query, args)
            rv = cur.fetchall()
            conn.commit()
    except Exception as e:
        print(f"Database query error: {e}", file=sys.stderr)
        rv = []
    finally:
        conn.close()
    return (rv[0] if rv else None) if one else rv

@app.route('/')
def index():
    # 1. Fetch KPI metrics (Latest run metrics instead of cumulative sum across history)
    latest_run = query_db("SELECT files_processed, files_failed FROM pipeline_runs ORDER BY started_at DESC LIMIT 1", one=True)
    if latest_run:
        total_files_processed = latest_run.get('files_processed', 0)
        total_files_failed = latest_run.get('files_failed', 0)
    else:
        total_files_processed = 0
        total_files_failed = 0

    total_flagged_res = query_db("SELECT COUNT(*) as val FROM standardized_records WHERE processing_status = 'flagged'", one=True)
    total_flagged = total_flagged_res.get('val') if total_flagged_res else 0

    total_records_res = query_db("SELECT COUNT(*) as val FROM standardized_records", one=True)
    total_records = total_records_res.get('val') if total_records_res else 0
```

```

total_runs_res = query_db("SELECT COUNT(*) as val FROM pipeline_runs", one=True)
total_runs = total_runs_res.get('val') if total_runs_res else 0

total_duplicates_res = query_db("SELECT COUNT(*) as val FROM standardized_records WHERE
duplicate_flag = TRUE", one=True)
total_duplicates = total_duplicates_res.get('val') if total_duplicates_res else 0

# 2. Fetch Recent Pipeline Runs
pipeline_runs = query_db("SELECT * FROM pipeline_runs ORDER BY started_at DESC LIMIT
10")

# 3. Fetch Clinic/Document Type Summary
summary_query = """
    SELECT
        COALESCE(hospital_name, 'Redacted / Unknown') as hospital_name,
        record_type,
        COUNT(*) as total_records,
        SUM(CASE WHEN processing_status = 'flagged' THEN 1 ELSE 0 END) as
flagged_count,
        SUM(CASE WHEN duplicate_flag = TRUE THEN 1 ELSE 0 END) as duplicate_count
    FROM standardized_records
    GROUP BY hospital_name, record_type
    """
clinic_summaries = query_db(summary_query)

return render_template(
    'index.html',
    total_files_processed=total_files_processed,
    total_files_failed=total_files_failed,
    total_flagged=total_flagged,
    total_records=total_records,
    total_runs=total_runs,
    total_duplicates=total_duplicates,
    pipeline_runs=pipeline_runs,
    clinic_summaries=clinic_summaries
)

@app.route('/records')
def records():
    search = request.args.get('search', '')
    rec_type = request.args.get('type', '')
    status = request.args.get('status', '')

    query = "SELECT * FROM standardized_records WHERE 1=1"
    params = []

    if search:
        query += " AND (patient_name LIKE %s OR document_id LIKE %s OR claim_no LIKE %s)"
        search_param = f"%{search}%"
        params.extend([search_param, search_param, search_param])

    if rec_type:
        query += " AND record_type = %s"
        params.append(rec_type)

```

```

    if status:
        if status == 'duplicate':
            query += " AND duplicate_flag = TRUE"
        elif status == 'processed':
            query += " AND processing_status = 'processed' AND (duplicate_flag = FALSE OR
duplicate_flag IS NULL)"
        else:
            query += " AND processing_status = %s"
            params.append(status)

    query += " ORDER BY processed_at DESC LIMIT 100"
    records_list = query_db(query, params)

    return render_template('records.html', records=records_list, search=search,
type=rec_type, status=status)

@app.route('/record/<record_id>')
def get_record(record_id):
    """Retrieve detailed properties of a single standardized record (returns JSON for
inspector)."""
    record = query_db("SELECT * FROM standardized_records WHERE id = %s", (record_id,),
one=True)
    if not record:
        return jsonify({'error': 'Record not found'}), 404
    return jsonify(record)

@app.route('/flagged')
def flagged():
    # Anomaly queue lists records where validation failed or results are out of bounds
    flagged_records = query_db("""
        SELECT * FROM standardized_records
        WHERE processing_status = 'flagged' OR test_analytics <> 'Within Range'
        ORDER BY processed_at DESC
    """)
    return render_template('flagged.html', records=flagged_records)

@app.route('/errors')
def errors():
    # Audit trail for failed files and JSON parsing errors
    ingestion_errors = query_db("SELECT * FROM ingestion_errors ORDER BY occurred_at DESC")
    return render_template('errors.html', errors=ingestion_errors)

@app.route('/run-pipeline', methods=['POST'])
def trigger_pipeline():
    """Trigger the ingestion and standardisation pipeline synchronously from the
dashboard."""
    try:
        run_pipeline(reset=True)
        return jsonify({'status': 'success', 'message': 'Pipeline run reset and completed
successfully!'})
    except Exception as e:
        return jsonify({'status': 'error', 'message': f"Pipeline run failed: {str(e)}"}),
500

```



```
@app.route('/clear-history', methods=['POST'])
def clear_history():
    """Clear all records from the pipeline_runs table."""
    conn = get_db_connection()
    try:
        with conn.cursor() as cur:
            cur.execute("SET FOREIGN_KEY_CHECKS = 0;")
            cur.execute("TRUNCATE TABLE pipeline_runs;")
            cur.execute("SET FOREIGN_KEY_CHECKS = 1;")
        conn.commit()
        return jsonify({'status': 'success', 'message': 'Pipeline run history cleared
successfully!'})
    except Exception as e:
        return jsonify({'status': 'error', 'message': f"Failed to clear history:
{str(e)}"}), 500
    finally:
        conn.close()

if __name__ == '__main__':
    # Pre-seed database with a fresh load on server start
    # We check WERKZEUG_RUN_MAIN to prevent the Flask reloader from running it twice in
debug mode
    if not app.debug or os.environ.get('WERKZEUG_RUN_MAIN') == 'true':
        try:
            print("Pre-seeding database with fresh load on server start...")
            run_pipeline(reset=True)
        except Exception as e:
            print(f"Failed to pre-seed database: {e}", file=sys.stderr)

    # Listen on localhost:5000
    app.run(debug=True, host='localhost', port=5000)
```

File: style.css

Path: ui\static\style.css

```
/* Veritas Claims Dashboard - Custom Vanilla CSS Design System */
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&family=JetBrains+Mono:wght@400;500&display=swap');

:root {
  --bg-main: #0f172a; /* slate-900 */
  --bg-surface: #1e293b; /* slate-800 */
  --bg-sidebar: #0b0f19; /* slate-950 */
  --border-color: #334155; /* slate-700 */
  --text-primary: #f8fafc; /* slate-50 */
  --text-secondary: #94a3b8; /* slate-400 */

  --primary: #6366f1; /* indigo-500 */
  --primary-hover: #4f46e5; /* indigo-600 */

  --success: #10b981; /* emerald-500 */
  --warning: #f59e0b; /* amber-500 */
  --danger: #ef4444; /* red-500 */
  --info: #3b82f6; /* blue-500 */
  --accent: #a855f7; /* purple-500 */
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: 'Inter', -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
  background-color: var(--bg-main);
  color: var(--text-primary);
  display: flex;
  min-height: 100vh;
  overflow-x: hidden;
}

/* Scrollbar Customization */
::-webkit-scrollbar {
  width: 8px;
  height: 8px;
}

::-webkit-scrollbar-track {
  background: var(--bg-main);
}

::-webkit-scrollbar-thumb {
```

```
        background: var(--border-color);
        border-radius: 4px;
    }
    ::-webkit-scrollbar-thumb:hover {
        background: var(--text-secondary);
    }

    /* Sidebar Layout */
    .sidebar {
        width: 260px;
        background-color: var(--bg-sidebar);
        border-right: 1px solid var(--border-color);
        padding: 24px;
        display: flex;
        flex-direction: column;
        position: fixed;
        height: 100vh;
        left: 0;
        top: 0;
        z-index: 100;
    }

    .sidebar .brand {
        font-size: 20px;
        font-weight: 700;
        color: var(--text-primary);
        display: flex;
        align-items: center;
        gap: 10px;
        margin-bottom: 36px;
        letter-spacing: 0.5px;
    }

    .sidebar .brand span {
        color: var(--primary);
    }

    .sidebar nav {
        display: flex;
        flex-direction: column;
        gap: 8px;
        flex-grow: 1;
    }

    .sidebar nav a {
        color: var(--text-secondary);
        text-decoration: none;
        padding: 12px 16px;
        border-radius: 8px;
        font-weight: 500;
        font-size: 14px;
        display: flex;
        align-items: center;
        gap: 12px;
    }
```

```
    transition: all 0.2s ease;
  }

.sidebar nav a:hover {
  color: var(--text-primary);
  background-color: rgba(99, 102, 241, 0.08);
}

.sidebar nav a.active {
  color: var(--text-primary);
  background-color: var(--primary);
}

/* Main Content Layout */
.main-wrapper {
  margin-left: 260px;
  padding: 32px;
  width: calc(100% - 260px);
  display: flex;
  flex-direction: column;
  gap: 32px;
}

/* Header */
.header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  border-bottom: 1px solid var(--border-color);
  padding-bottom: 20px;
}

.header h1 {
  font-size: 26px;
  font-weight: 700;
  letter-spacing: -0.5px;
}

.header p {
  color: var(--text-secondary);
  font-size: 14px;
  margin-top: 4px;
}

/* Action Button */
.btn {
  background-color: var(--primary);
  color: var(--text-primary);
  border: none;
  padding: 10px 20px;
  border-radius: 8px;
  font-weight: 600;
  font-size: 14px;
  cursor: pointer;
}
```

```
    display: flex;
    align-items: center;
    gap: 8px;
    transition: all 0.2s ease;
    box-shadow: 0 4px 12px rgba(99, 102, 241, 0.2);
  }

  .btn:hover {
    background-color: var(--primary-hover);
    transform: translateY(-1px);
  }

  .btn:active {
    transform: translateY(1px);
  }

  .btn-secondary {
    background-color: transparent;
    border: 1px solid var(--border-color);
    color: var(--text-primary);
    box-shadow: none;
  }

  .btn-secondary:hover {
    background-color: var(--bg-surface);
  }

  /* KPI Summary Cards Grid */
  .kpi-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
    gap: 20px;
  }

  .kpi-card {
    background-color: var(--bg-surface);
    border: 1px solid var(--border-color);
    border-radius: 12px;
    padding: 24px;
    display: flex;
    flex-direction: column;
    gap: 10px;
    transition: transform 0.2s ease, box-shadow 0.2s ease;
  }

  .kpi-card:hover {
    transform: translateY(-2px);
    box-shadow: 0 8px 24px rgba(0, 0, 0, 0.2);
  }

  .kpi-card .card-title {
    font-size: 13px;
    font-weight: 600;
    color: var(--text-secondary);
  }
```

```
    text-transform: uppercase;
    letter-spacing: 0.5px;
}

.kpi-card .card-value {
    font-size: 32px;
    font-weight: 700;
    letter-spacing: -1px;
}

.kpi-card.failed {
    border-left: 4px solid var(--danger);
}

.kpi-card.processed {
    border-left: 4px solid var(--success);
}

.kpi-card.flagged {
    border-left: 4px solid var(--warning);
}

.kpi-card.info {
    border-left: 4px solid var(--info);
}

/* General Layout Card Section */
.card-section {
    background-color: var(--bg-surface);
    border: 1px solid var(--border-color);
    border-radius: 12px;
    padding: 24px;
    display: flex;
    flex-direction: column;
    gap: 20px;
}

.card-section h2 {
    font-size: 18px;
    font-weight: 600;
    border-bottom: 1px solid var(--border-color);
    padding-bottom: 12px;
    margin-bottom: 4px;
}

/* Dynamic Tables styling */
.table-container {
    overflow-x: auto;
    width: 100%;
}

table {
    width: 100%;
    border-collapse: collapse;
    text-align: left;
    font-size: 14px;
}
```

```
thead th {
  background-color: rgba(15, 23, 42, 0.4);
  color: var(--text-secondary);
  font-weight: 600;
  padding: 16px;
  border-bottom: 1px solid var(--border-color);
  position: sticky;
  top: 0;
}

tbody tr {
  border-bottom: 1px solid rgba(51, 65, 85, 0.5);
  transition: background-color 0.15s ease;
}

tbody tr:hover {
  background-color: rgba(255, 255, 255, 0.02);
}

tbody td {
  padding: 16px;
  color: var(--text-primary);
}

/* Badges styling */
.badge {
  display: inline-flex;
  align-items: center;
  padding: 4px 10px;
  border-radius: 50px;
  font-size: 11px;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 0.5px;
}

.badge-processed {
  background-color: rgba(16, 185, 129, 0.15);
  color: var(--success);
}

.badge-flagged {
  background-color: rgba(245, 158, 11, 0.15);
  color: var(--warning);
}

.badge-duplicate {
  background-color: rgba(59, 130, 246, 0.15);
  color: var(--info);
}

.badge-error {
  background-color: rgba(239, 68, 68, 0.15);
```

```
        color: var(--danger);
    }

    /* Test Analytics Range Flags styling */
    .range-badge {
        display: inline-flex;
        padding: 2px 8px;
        border-radius: 4px;
        font-size: 11px;
        font-weight: 500;
    }

    .range-within {
        background-color: rgba(16, 185, 129, 0.1);
        color: var(--success);
    }
    .range-above {
        background-color: rgba(239, 68, 68, 0.1);
        color: var(--danger);
    }
    .range-below {
        background-color: rgba(168, 85, 247, 0.1);
        color: var(--accent);
    }
    .range-outlier {
        background-color: rgba(239, 68, 68, 0.2);
        color: var(--danger);
        font-weight: 700;
        border: 1px solid var(--danger);
    }
    .range-invalid {
        background-color: rgba(245, 158, 11, 0.1);
        color: var(--warning);
    }

    /* Filter controls and Search Bar */
    .filter-form {
        display: flex;
        flex-wrap: wrap;
        gap: 16px;
        align-items: center;
        background-color: rgba(15, 23, 42, 0.3);
        padding: 16px;
        border-radius: 8px;
        border: 1px solid var(--border-color);
    }

    .form-group {
        display: flex;
        flex-direction: column;
        gap: 6px;
    }

    .form-group label {
```



```
    font-size: 11px;
    font-weight: 600;
    color: var(--text-secondary);
    text-transform: uppercase;
}

.form-input {
    background-color: var(--bg-main);
    border: 1px solid var(--border-color);
    color: var(--text-primary);
    padding: 8px 12px;
    border-radius: 6px;
    font-size: 14px;
    min-width: 180px;
    transition: border-color 0.2s ease;
}

.form-input:focus {
    outline: none;
    border-color: var(--primary);
}

/* Split Section for Dashboard */
.split-sections {
    display: grid;
    grid-template-columns: 2fr 1fr;
    gap: 32px;
}

@media (max-width: 1024px) {
    .split-sections {
        grid-template-columns: 1fr;
    }
}

/* Detailed Audit Viewer/Pre */
pre.raw-payload {
    background-color: var(--bg-main);
    border: 1px solid var(--border-color);
    padding: 16px;
    border-radius: 8px;
    font-family: 'JetBrains Mono', monospace;
    font-size: 12px;
    color: #e2e8f0;
    overflow-x: auto;
    white-space: pre-wrap;
    max-height: 400px;
}

/* Interactive Modal Details Panel */
.modal-overlay {
    display: none;
    position: fixed;
    top: 0;
```

```
    left: 0;
    width: 100vw;
    height: 100vh;
    background-color: rgba(15, 23, 42, 0.85);
    z-index: 1000;
    justify-content: center;
    align-items: center;
    backdrop-filter: blur(4px);
}

.modal-content {
    background-color: var(--bg-surface);
    border: 1px solid var(--border-color);
    border-radius: 12px;
    width: 90%;
    max-width: 800px;
    max-height: 85vh;
    display: flex;
    flex-direction: column;
    box-shadow: 0 20px 25px -5px rgba(0, 0, 0, 0.3);
    animation: modal-fade-in 0.25s cubic-bezier(0.16, 1, 0.3, 1);
}

@keyframes modal-fade-in {
    from { transform: translateY(10px); opacity: 0; }
    to { transform: translateY(0); opacity: 1; }
}

.modal-header {
    padding: 20px 24px;
    border-bottom: 1px solid var(--border-color);
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.modal-header h3 {
    font-size: 18px;
    font-weight: 600;
}

.modal-close {
    background: transparent;
    border: none;
    color: var(--text-secondary);
    font-size: 24px;
    cursor: pointer;
}

.modal-close:hover {
    color: var(--text-primary);
}

.modal-body {
```

```
padding: 24px;
overflow-y: auto;
display: flex;
flex-direction: column;
gap: 20px;
}

.modal-footer {
padding: 20px 24px;
border-top: 1px solid var(--border-color);
display: flex;
justify-content: flex-end;
}

/* Inspector details layout */
.inspector-section {
display: flex;
flex-direction: column;
gap: 8px;
}

.inspector-section-title {
font-size: 12px;
font-weight: 600;
text-transform: uppercase;
color: var(--text-secondary);
letter-spacing: 0.5px;
margin-bottom: 4px;
}

.inspector-grid {
display: grid;
grid-template-columns: repeat(2, 1fr);
gap: 16px;
background-color: rgba(15, 23, 42, 0.3);
padding: 16px;
border-radius: 8px;
border: 1px solid var(--border-color);
}

.inspector-item {
display: flex;
flex-direction: column;
gap: 4px;
}

.inspector-label {
font-size: 11px;
color: var(--text-secondary);
}

.inspector-value {
font-size: 14px;
font-weight: 500;
```

```
}

/* Side-by-Side Comparison tables */
.comparison-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 20px;
}

.comparison-pane {
  display: flex;
  flex-direction: column;
  gap: 8px;
}

.comparison-pane h4 {
  font-size: 13px;
  color: var(--text-secondary);
  text-transform: uppercase;
  letter-spacing: 0.5px;
}

.comparison-box {
  background-color: var(--bg-main);
  border: 1px solid var(--border-color);
  border-radius: 8px;
  padding: 16px;
  min-height: 80px;
  font-size: 14px;
  display: flex;
  align-items: center;
}

.comparison-highlight {
  background-color: rgba(99, 102, 241, 0.1);
  border: 1px solid var(--primary);
}
```

File: errors.html

Path: ui\templates\errors.html

```
{% extends 'layout.html' %}

{% block content %}
<div class="header">
    <div>
        <h1>Ingestion Errors & Failures</h1>
        <p>Audit trail of malformed JSON structures, invalid document payloads, or database
load conflicts.</p>
    </div>
</div>

<!-- Errors List -->
<div class="card-section">
    <div class="table-container">
        <table>
            <thead>
                <tr>
                    <th>Error ID</th>
                    <th>File Reference</th>
                    <th>Stage</th>
                    <th>Failure Reason</th>
                    <th>Timestamp</th>
                    <th>Actions</th>
                </tr>
            </thead>
            <tbody>
                {% for err in errors %}
                <tr>
                    <td><strong>#{ err.error_id }</strong></td>
                    <td>
                        <div style="font-family: 'JetBrains Mono', monospace; font-size:
13px;">{{ err.file_path.replace('\\', '/').split('/')[ -1 ] }}</div>
                        <div style="font-size: 11px; color: var(--text-secondary);">{{
err.file_path }}</div>
                    </td>
                    <td><span class="badge badge-error">{{ err.stage | upper }}</span></td>
                    <td>
                        <div style="color: var(--danger); font-weight: 500;">{{
err.error_reason }}</div>
                    </td>
                    <td>{{ err.occurred_at }}</td>
                    <td>
                        <button class="btn btn-secondary" style="padding: 6px 12px; font-
size: 12px;" onclick="viewPayload({{ err.error_id }}, '{{ err.file_path.replace('\\',
'/').split('/')[ -1 ] }})'>
                            <i data-lucide="file-text"></i> Audit Payload
                        </button>
                    </td>
                </tr>
                {% endfor %}
            </tbody>
        </table>
    </div>
</div>
```

```

        </td>
      </tr>
    {% else %}
    <tr>
      <td colspan="6" style="text-align: center; color: var(--text-secondary); padding: 32px;">
        <i data-lucide="check-circle2" style="color: var(--success); width: 36px; height: 36px; margin-bottom: 8px;"></i>
        <div>No ingestion errors logged. Ingestion is 100% clean!</div>
      </td>
    </tr>
  {% endfor %}
</tbody>
</table>
</div>
</div>

<!-- Raw Payload Overlay Modal -->
<div id="payload-modal" class="modal-overlay">
  <div class="modal-content" style="max-width: 900px;">
    <div class="modal-header">
      <h3 id="payload-title">Raw Payload Audit Trail</h3>
      <button class="modal-close" onclick="closePayload()">&times;</button>
    </div>
    <div class="modal-body">
      <div class="inspector-section">
        <span class="inspector-section-title">Ingested Content (Non-conforming
JSON)</span>
        <pre class="raw-payload" id="payload-pre"></pre>
      </div>
    </div>
    <div class="modal-footer">
      <button class="btn btn-secondary" onclick="closePayload()">Close
Viewer</button>
    </div>
  </div>
</div>

<!-- Helper data block for python errors object conversion -->
<script>
  const errorPayloads = {
    {% for err in errors %}
    "{{ err.error_id }}": {{ err.raw_payload | tojson }},
    {% endfor %}
  };

  function viewPayload(errorId, filename) {
    document.getElementById('payload-title').innerText = `Raw Payload Audit:
${filename}`;
    document.getElementById('payload-pre').innerText = errorPayloads[errorId] || "No
raw payload stored.";
    document.getElementById('payload-modal').style.display = 'flex';
    lucide.createIcons();
  }

```

```
function closePayload() {  
    document.getElementById('payload-modal').style.display = 'none';  
}  
</script>  
{% endblock %}
```

File: flagged.html

Path: ui\templates\flagged.html

```
{% extends 'layout.html' %}

{% block content %}
<div class="header">
    <div>
        <h1>Flagged Anomalies Queue</h1>
        <p>Review and audit clinical records that triggered outlier bounds, abnormal
reference ranges, or parsing warnings.</p>
    </div>
</div>

<!-- Anomaly Queue Grid -->
<div class="card-section">
    <div class="table-container">
        <table>
            <thead>
                <tr>
                    <th>Claim Number</th>
                    <th>Patient Name</th>
                    <th>Test / Medicine Name</th>
                    <th>Result Text</th>
                    <th>Parsed Value</th>
                    <th>Normal Reference Bounds</th>
                    <th>Anomaly Deviation</th>
                    <th>Warning Details</th>
                    <th>Actions</th>
                </tr>
            </thead>
            <tbody>
                {% for r in records %}
                <tr>
                    <td><strong>{{ r.claim_no }}</strong></td>
                    <td>{{ r.patient_name }}</td>
                    <td>
                        <div style="font-weight: 600;">{{ r.test_name_canonical or
r.medicine }}</div>
                        <div style="font-size: 11px; color: var(--text-secondary); text-
transform: uppercase;">{{ r.record_type }}</div>
                    </td>
                    <td><code style="font-family: 'JetBrains Mono', monospace;">{{
r.result_text or 'N/A' }}</code></td>
                    <td><strong>{{ r.result_value if r.result_value is not none else 'None'
}}</strong></td>
                    <td>
                        {% if r.range_low is not none or r.range_high is not none %}
                        {{ r.range_low if r.range_low is not none else 'None' }} - {{
r.range_high if r.range_high is not none else 'None' }}

```



```

        {% else %}
        {{ r.range_text or 'None' }}
        {% endif %}
        <span style="font-size: 11px; color: var(--text-secondary);">{{
r.unit_canonical or r.unit_original }}</span>
    </td>
    <td>
        {% if r.test_analytics == 'Outlier' %}
        <span class="range-badge range-outlier">OUTLIER (EXTREME)</span>
        {% elif r.test_analytics == 'Above Range' %}
        <span class="range-badge range-above">HIGH</span>
        {% elif r.test_analytics == 'Below Range' %}
        <span class="range-badge range-below">LOW</span>
        {% elif r.test_analytics == 'Invalid' %}
        <span class="range-badge range-invalid">PARSING ERROR</span>
        {% else %}
        <span class="range-badge range-invalid">FLAGGED</span>
        {% endif %}
    </td>
    <td>
        <span style="font-size: 12px; color: var(--text-secondary);">
            {{ r.error_message or 'Abnormal threshold boundary deviation.'
}}
        </span>
    </td>
    <td>
        <button class="btn btn-secondary" style="padding: 6px 12px; font-
size: 12px;" onclick="openInspector('{{ r.id }}')">
            <i data-lucide="eye"></i> Inspect
        </button>
    </td>
</tr>
{% else %}
<tr>
    <td colspan="9" style="text-align: center; color: var(--text-
secondary); padding: 32px;">
        <i data-lucide="check-circle" style="color: var(--success); width:
36px; height: 36px; margin-bottom: 8px;"></i>
        <div>Flagged queue is empty. All ingested claims are within safe
thresholds!</div>
    </td>
</tr>
{% endfor %}
</tbody>
</table>
</div>
</div>

<!-- Copy of the inspector modal structure for consistency -->
<div id="inspector-modal" class="modal-overlay">
    <div class="modal-content">
        <div class="modal-header">
            <h3>Standardised Record Inspector</h3>
            <button class="modal-close" onclick="closeInspector()">&times;</button>

```

```

</div>
<div class="modal-body">
  <div class="inspector-section">
    <span class="inspector-section-title">Demographics & Origin</span>
    <div class="inspector-grid">
      <div class="inspector-item"><span class="inspector-label">Patient
Name</span><span class="inspector-value" id="ins-patient-name"></span></div>
      <div class="inspector-item"><span class="inspector-label">Age /
Gender</span><span class="inspector-value" id="ins-age-gender"></span></div>
      <div class="inspector-item"><span class="inspector-label">Hospital /
Lab</span><span class="inspector-value" id="ins-hospital"></span></div>
      <div class="inspector-item"><span class="inspector-label">Attending
Doctor</span><span class="inspector-value" id="ins-doctor"></span></div>
      <div class="inspector-item"><span class="inspector-label">Claim
Number</span><span class="inspector-value" id="ins-claim-no"></span></div>
      <div class="inspector-item"><span class="inspector-label">Document
ID</span><span class="inspector-value" id="ins-doc-id"></span></div>
    </div>
  </div>

  <div class="inspector-section">
    <span class="inspector-section-title">Standardisation & Resolution
mapping</span>
    <div class="comparison-grid">
      <div class="comparison-pane">
        <h4>Original Raw Values</h4>
        <div class="comparison-box" id="ins-raw-values"></div>
      </div>
      <div class="comparison-pane">
        <h4>Standardised Canonical Values</h4>
        <div class="comparison-box comparison-highlight" id="ins-std-
values"></div>
      </div>
    </div>
  </div>

  <div class="inspector-section">
    <span class="inspector-section-title">Normalisation Confidence</span>
    <div class="inspector-grid" style="background-color: rgba(99, 102, 241,
0.04); border-color: var(--primary);">
      <div class="inspector-item"><span class="inspector-label">Resolution
Method</span><span class="inspector-value" id="ins-method"></span></div>
      <div class="inspector-item"><span class="inspector-label">Match Score /
Confidence</span><span class="inspector-value" id="ins-confidence"></span></div>
    </div>
  </div>

  <div class="inspector-section">
    <span class="inspector-section-title">Audit Log (78 columns full row
schema)</span>
    <pre class="raw-payload" id="ins-full-json"></pre>
  </div>
</div>
<div class="modal-footer">

```

```

        <button class="btn btn-secondary" onclick="closeInspector()">Close
Inspector</button>
    </div>
</div>
</div>

<script>
    // Copy JavaScript controller from records.html
    function openInspector(recordId) {
        fetch(`/record/${recordId}`)
            .then(res => res.json())
            .then(data => {
                document.getElementById('ins-patient-name').innerText = data.patient_name
                || 'N/A';
                document.getElementById('ins-age-gender').innerText = `${data.age || 'N/A'}
/ ${data.gender || 'N/A'}`;
                document.getElementById('ins-hospital').innerText = data.hospital_name ||
'N/A';
                document.getElementById('ins-doctor').innerText = data.doctor_name ||
'N/A';
                document.getElementById('ins-claim-no').innerText = data.claim_no || 'N/A';
                document.getElementById('ins-doc-id').innerText = data.document_id ||
'N/A';

                if (data.record_type === 'lab_report') {
                    document.getElementById('ins-raw-values').innerHTML = `
                        <div>
                            <div><strong>Test Name:</strong> ${data.test_name_original ||
'N/A'}</div>
                            <div><strong>Result:</strong> ${data.result_text || 'N/A'}
</div>
                            <div><strong>Unit:</strong> ${data.unit_original || 'N/A'}
</div>
                            <div><strong>Range:</strong> ${data.range_text || 'N/A'}</div>
                        </div>
                    `;
                    document.getElementById('ins-std-values').innerHTML = `
                        <div>
                            <div><strong>Canonical Test:</strong>
${data.test_name_canonical || 'N/A'}</div>
                            <div><strong>Value:</strong> ${data.result_value !== null ?
data.result_value : 'N/A'}</div>
                            <div><strong>Canonical Unit:</strong> ${data.unit_canonical ||
'N/A'}</div>
                            <div><strong>Parsed Range:</strong> ${data.range_low !== null ?
data.range_low : 'None'} - ${data.range_high !== null ? data.range_high : 'None'}</div>
                            <div style="margin-top: 4px;">
                                <strong>Analytics Flag:</strong>
                                <span class="range-badge range-${
                                    {'Within Range': 'within', 'Above Range': 'above',
'Below Range': 'below', 'Outlier': 'outlier', 'Invalid': 'invalid'}[data.test_analytics] ||
'invalid'
                                }">${data.test_analytics}</span>
                            </div>
                        </div>
                    `;
                }
            });
    }

```

```

        </div>
        `;
    } else {
        document.getElementById('ins-raw-values').innerHTML = `
            <div>
                <div><strong>Medicine:</strong> ${data.medication_medicine ||
data.medication_name || 'N/A'}</div>
                <div><strong>Dose:</strong> ${data.medication_dose || 'N/A'}
</div>
                <div><strong>Frequency:</strong> ${data.medication_frequency ||
'N/A'}</div>
            </div>
        `;
        document.getElementById('ins-std-values').innerHTML = `
            <div>
                <div><strong>Canonical Name:</strong> ${data.medicine || 'N/A'}
</div>
                <div><strong>Dosage Form:</strong> ${data.medicine_type ||
'N/A'}</div>
                <div><strong>Dose:</strong> ${data.dose || 'N/A'}</div>
                <div><strong>Frequency:</strong> ${data.frequency || 'N/A'}
</div>
            </div>
        `;
    }

    document.getElementById('ins-method').innerText = data.normalization_method
? data.normalization_method.replace('_', ' ').toUpperCase() : 'N/A';
    document.getElementById('ins-confidence').innerText =
data.normalization_confidence !== null ? `${Math.round(data.normalization_confidence *
100)}%` : 'N/A';
    document.getElementById('ins-full-json').innerText = JSON.stringify(data,
null, 4);

    document.getElementById('inspector-modal').style.display = 'flex';
    lucide.createIcons();
})
.catch(err => {
    alert("Failed to load record details.");
    console.error(err);
});
}

function closeInspector() {
    document.getElementById('inspector-modal').style.display = 'none';
}
</script>
{% endblock %}

```

File: index.html

Path: ui\templates\index.html

```
{% extends 'layout.html' %}

{% block content %}
<div class="header">
    <div>
        <h1>Operational Dashboard</h1>
        <p>Real-time monitor for medical record ingestion, deduplication, and anomaly
classification.</p>
    </div>
    <button id="pipeline-btn" class="btn" onclick="triggerPipeline()">
        <i data-lucide="refresh-cw"></i> Run Pipeline
    </button>
</div>

<!-- KPI Summary Section -->
<div class="kpi-grid">
    <div class="kpi-card info">
        <span class="card-title">Total Records Ingested</span>
        <span class="card-value">{{ total_records }}</span>
    </div>
    <div class="kpi-card processed">
        <span class="card-title">Total Files Processed</span>
        <span class="card-value">{{ total_files_processed }}</span>
    </div>
    <div class="kpi-card failed">
        <span class="card-title">Ingestion Failures</span>
        <span class="card-value">{{ total_files_failed }}</span>
    </div>
    <div class="kpi-card flagged">
        <span class="card-title">Flagged Anomalies</span>
        <span class="card-value">{{ total_flagged }}</span>
    </div>
    <div class="kpi-card info">
        <span class="card-title">Duplicate Rows</span>
        <span class="card-value">{{ total_duplicates }}</span>
    </div>
</div>

<div class="split-sections">
    <div class="card-section">
        <div style="display: flex; align-items: center; gap: 12px; border-bottom: 1px solid
var(--border-color); padding-bottom: 8px; margin-bottom: 12px;">
            <button class="btn btn-secondary" style="padding: 6px 12px; font-size: 12px;
background-color: rgba(239, 68, 68, 0.1); color: var(--danger); border-color: rgba(239, 68,
68, 0.2);" onclick="clearHistory()">
                <i data-lucide="trash-2"></i> Clear History
            </button>
        </div>
    </div>
</div>
```

```

    <h2 style="border: none; margin: 0; padding: 0;">Pipeline Run History</h2>
  </div>
  <div class="table-container">
    <table>
      <thead>
        <tr>
          <th>Run ID</th>
          <th>Started At</th>
          <th>Finished At</th>
          <th>Files Seen</th>
          <th>Processed</th>
          <th>Failed</th>
          <th>Flagged</th>
          <th>Duplicates</th>
        </tr>
      </thead>
      <tbody>
        {% for run in pipeline_runs %}
        <tr>
          <td><strong>#{ run.run_id }</strong></td>
          <td>{{ run.started_at }}</td>
          <td>{{ run.finished_at or 'Running...' }}</td>
          <td>{{ run.files_seen }}</td>
          <td>{{ run.files_processed }}</td>
          <td>
            {% if run.files_failed > 0 %}
            <span class="badge badge-error">{{ run.files_failed }}</span>
            {% else %}
            0
            {% endif %}
          </td>
          <td>
            {% if run.records_flagged > 0 %}
            <span class="badge badge-flagged">{{ run.records_flagged }}
</span>
            {% else %}
            0
            {% endif %}
          </td>
          <td>{{ run.duplicates_skipped }}</td>
        </tr>
        {% else %}
        <tr>
          <td colspan="8" style="text-align: center; color: var(--text-secondary);">No pipeline runs executed yet.</td>
        </tr>
        {% endfor %}
      </tbody>
    </table>
  </div>
</div>

<!-- Clinic / Document Type Summary -->
<div class="card-section">

```

```

<h2>Clinic & Doc-Type Groupings</h2>
<div class="table-container">
  <table>
    <thead>
      <tr>
        <th>Entity / Document Type</th>
        <th>Count</th>
        <th>Flagged</th>
      </tr>
    </thead>
    <tbody>
      {% for item in clinic_summaries %}
      <tr>
        <td>
          <div style="font-weight: 600;">{{ item.hospital_name }}</div>
          <div style="font-size: 11px; color: var(--text-secondary);
text-transform: uppercase;">{{ item.record_type }}</div>
        </td>
        <td>{{ item.total_records }}</td>
        <td>
          {% if item.flagged_count > 0 %}
          <span class="badge badge-flagged">{{ item.flagged_count }}
</span>
          {% else %}
          0
          {% endif %}
        </td>
      </tr>
      {% else %}
      <tr>
        <td colspan="3" style="text-align: center; color: var(--text-
secondary);">No records standardise-loaded.</td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
</div>
</div>
<script>
  function clearHistory() {
    if (confirm("Are you sure you want to clear the entire pipeline run history?
This cannot be undone.)) {
      fetch('/clear-history', { method: 'POST' })
        .then(response => response.json())
        .then(data => {
          alert(data.message);
          window.location.reload();
        })
        .catch(err => {
          alert("Failed to clear history.");
          console.error(err);
        });
    }
  }
}

```

```
    }  
  </script>  
{% endblock %}
```


File: layout.html

Path: ui\templates\layout.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Veritas Claims Operational Dashboard</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
  <!-- Lucide Icons CDN -->
  <script src="https://unpkg.com/lucide@latest"></script>
</head>
<body>

  <!-- Sidebar Navigation -->
  <div class="sidebar">
    <div class="brand">
      <i data-lucide="shield-check"></i>
      Veritas <span>Claims</span>
    </div>
    <nav>
      <a href="/" id="nav-home" class="nav-item"><i data-lucide="layout-dashboard">
</i>Dashboard</a>
      <a href="/records" id="nav-records" class="nav-item"><i data-lucide="database">
</i>Standardised Records</a>
      <a href="/flagged" id="nav-flagged" class="nav-item"><i data-lucide="alert-
octagon"></i>Flagged Queue</a>
      <a href="/errors" id="nav-errors" class="nav-item"><i data-lucide="file-
warning"></i>Ingestion Errors</a>
    </nav>
  </div>

  <!-- Main Workspace -->
  <div class="main-wrapper">
    <!-- Page Injection -->
    {% block content %}{% endblock %}
  </div>

  <!-- Global Scripts -->
  <script>
    // Initialize Lucide Icons
    lucide.createIcons();

    // Active Navigation Marker logic
    const path = window.location.pathname;
    document.querySelectorAll('.nav-item').forEach(link => {
      if (link.getAttribute('href') === path) {
        link.classList.add('active');
      } else {
```

```
        link.classList.remove('active');
    }
});

// Global Pipeline Trigger Handler
function triggerPipeline() {
    const btn = document.getElementById('pipeline-btn');
    const originalContent = btn.innerHTML;
    btn.disabled = true;
    btn.innerHTML = '<i data-lucide="loader" class="animate-spin"></i> Running  
Ingestion...';
    lucide.createIcons();

    fetch('/run-pipeline', { method: 'POST' })
        .then(response => response.json())
        .then(data => {
            alert(data.message);
            window.location.reload();
        })
        .catch(err => {
            alert("Pipeline execution failed. Check console logs.");
            console.error(err);
            btn.disabled = false;
            btn.innerHTML = originalContent;
            lucide.createIcons();
        });
}
</script>
</body>
</html>
```

File: records.html

Path: ui\templates\records.html

```
{% extends 'layout.html' %}

{% block content %}
<div class="header">
    <div>
        <h1>Standardised Claims Records</h1>
        <p>Browse and audit structured medical reports stored in the unified database
schema.</p>
    </div>
</div>

<!-- Search & Filtering Controls -->
<form class="filter-form" method="GET" action="/records">
    <div class="form-group">
        <label for="search">Search Input</label>
        <input type="text" id="search" name="search" class="form-input" placeholder="Name,
Document ID, Claim No..." value="{{ search }}">
    </div>

    <div class="form-group">
        <label for="type">Record Type</label>
        <select id="type" name="type" class="form-input">
            <option value="">All Document Types</option>
            <option value="discharge_summary" {% if type == 'discharge_summary' %}selected{% endif %}>Discharge Summary</option>
            <option value="lab_report" {% if type == 'lab_report' %}selected{% endif %}>Lab
Report</option>
        </select>
    </div>

    <div class="form-group">
        <label for="status">Audit Status</label>
        <select id="status" name="status" class="form-input">
            <option value="">All Statuses</option>
            <option value="processed" {% if status == 'processed' %}selected{% endif %}>Processed</option>
            <option value="flagged" {% if status == 'flagged' %}selected{% endif %}>Flagged
(Anomalies)</option>
            <option value="duplicate" {% if status == 'duplicate' %}selected{% endif %}>Duplicates</option>
        </select>
    </div>

    <div class="form-group" style="align-self: flex-end;">
        <button type="submit" class="btn"><i data-lucide="search"></i> Apply
Filters</button>
    </div>
</form>
</div>
```

```

    <div class="form-group" style="align-self: flex-end;">
      <a href="/records" class="btn btn-secondary"><i data-lucide="x-circle"></i>
Clear</a>
    </div>
</form>

<!-- Records Grid -->
<div class="card-section">
  <div class="table-container">
    <table>
      <thead>
        <tr>
          <th>Claim Number</th>
          <th>Document ID</th>
          <th>Record Type</th>
          <th>Patient Name</th>
          <th>Entity Name</th>
          <th>Target Field / Identifier</th>
          <th>Status</th>
          <th>Actions</th>
        </tr>
      </thead>
      <tbody>
        {% for r in records %}
          <tr>
            <td><strong>{{ r.claim_no }}</strong></td>
            <td>{{ r.document_id }}</td>
            <td><span style="font-size: 12px; font-weight: 500; text-transform:
uppercase;">{{ r.record_type }}</span></td>
            <td>{{ r.patient_name }}</td>
            <td>{{ r.hospital_name }}</td>
            <td>
              {% if r.record_type == 'lab_report' %}
                <div style="font-weight: 600; color: var(--accent);">{{
r.test_name_canonical or r.test_name_original }}</div>
                <div style="font-size: 11px; color: var(--text-secondary);">Result:
{{ r.result_text }}</div>
              {% else %}
                <div style="font-weight: 600; color: var(--info);">{{ r.medicine }}
</div>
                <div style="font-size: 11px; color: var(--text-secondary);">Dose:
{{ r.dose }} ({{ r.frequency }})</div>
              {% endif %}
            </td>
            <td>
              {% if r.duplicate_flag %}
                <span class="badge badge-duplicate">Duplicate</span>
              {% elif r.processing_status == 'flagged' %}
                <span class="badge badge-flagged">Flagged</span>
              {% else %}
                <span class="badge badge-processed">Processed</span>
              {% endif %}
            </td>
            <td>

```

```

        <button class="btn btn-secondary" style="padding: 6px 12px; font-size: 12px;" onclick="openInspector('{{ r.id }}')">
            <i data-lucide="eye"></i> Inspect
        </button>
    </td>
</tr>
{% else %}
<tr>
    <td colspan="8" style="text-align: center; color: var(--text-secondary);">No records match the filter criteria.</td>
</tr>
{% endfor %}
</tbody>
</table>
</div>
</div>

<!-- Interactive Record Inspector Overlay Modal -->
<div id="inspector-modal" class="modal-overlay">
    <div class="modal-content">
        <div class="modal-header">
            <h3>Standardised Record Inspector</h3>
            <button class="modal-close" onclick="closeInspector()">&times;</button>
        </div>
        <div class="modal-body">
            <!-- Part 1: Metadata Section -->
            <div class="inspector-section">
                <span class="inspector-section-title">Demographics & Origin</span>
                <div class="inspector-grid">
                    <div class="inspector-item"><span class="inspector-label">Patient Name</span><span class="inspector-value" id="ins-patient-name"></span></div>
                    <div class="inspector-item"><span class="inspector-label">Age / Gender</span><span class="inspector-value" id="ins-age-gender"></span></div>
                    <div class="inspector-item"><span class="inspector-label">Hospital / Lab</span><span class="inspector-value" id="ins-hospital"></span></div>
                    <div class="inspector-item"><span class="inspector-label">Attending Doctor</span><span class="inspector-value" id="ins-doctor"></span></div>
                    <div class="inspector-item"><span class="inspector-label">Claim Number</span><span class="inspector-value" id="ins-claim-no"></span></div>
                    <div class="inspector-item"><span class="inspector-label">Document ID</span><span class="inspector-value" id="ins-doc-id"></span></div>
                </div>
            </div>

            <!-- Part 2: Normalisation Comparison -->
            <div class="inspector-section">
                <span class="inspector-section-title">Standardisation & Resolution mapping</span>
                <div class="comparison-grid">
                    <!-- Left: Source Field Value -->
                    <div class="comparison-pane">
                        <h4>Original Raw Values</h4>
                        <div class="comparison-box" id="ins-raw-values"></div>
                    </div>

```

```

        <!-- Right: Target Standard Value -->
        <div class="comparison-pane">
            <h4>Standardised Canonical Values</h4>
            <div class="comparison-box comparison-highlight" id="ins-std-
values"></div>
        </div>
    </div>
</div>

<!-- Part 3: Metrics of Mapping Resolution -->
<div class="inspector-section">
    <span class="inspector-section-title">Normalisation Confidence</span>
    <div class="inspector-grid" style="background-color: rgba(99, 102, 241,
0.04); border-color: var(--primary);">
        <div class="inspector-item"><span class="inspector-label">Resolution
Method</span><span class="inspector-value" id="ins-method"></span></div>
        <div class="inspector-item"><span class="inspector-label">Match Score /
Confidence</span><span class="inspector-value" id="ins-confidence"></span></div>
    </div>
</div>

<!-- Part 4: Complete schema schema rows expansion -->
<div class="inspector-section">
    <span class="inspector-section-title">Audit Log (78 columns full row
schema)</span>
    <pre class="raw-payload" id="ins-full-json"></pre>
</div>
</div>
<div class="modal-footer">
    <button class="btn btn-secondary" onclick="closeInspector()">Close
Inspector</button>
</div>
</div>
</div>

<script>
    function openInspector(recordId) {
        // Fetch detailed record from server
        fetch(`/record/${recordId}`)
            .then(res => res.json())
            .then(data => {
                // Demographics
                document.getElementById('ins-patient-name').innerText = data.patient_name
                || 'N/A';
                document.getElementById('ins-age-gender').innerText = `${data.age || 'N/A'}
/ ${data.gender || 'N/A'}`;
                document.getElementById('ins-hospital').innerText = data.hospital_name ||
'N/A';
                document.getElementById('ins-doctor').innerText = data.doctor_name ||
'N/A';
                document.getElementById('ins-claim-no').innerText = data.claim_no || 'N/A';
                document.getElementById('ins-doc-id').innerText = data.document_id ||
'N/A';
            })
    }

```

```

// Details based on record type
if (data.record_type === 'lab_report') {
    document.getElementById('ins-raw-values').innerHTML = `
        <div>
            <div><strong>Test Name:</strong> ${data.test_name_original ||
'N/A'}</div>
            <div><strong>Result:</strong> ${data.result_text || 'N/A'}
</div>
            <div><strong>Unit:</strong> ${data.unit_original || 'N/A'}
</div>
            <div><strong>Range:</strong> ${data.range_text || 'N/A'}</div>
        </div>
`;
    document.getElementById('ins-std-values').innerHTML = `
        <div>
            <div><strong>Canonical Test:</strong>
${data.test_name_canonical || 'N/A'}</div>
            <div><strong>Value:</strong> ${data.result_value !== null ?
data.result_value : 'N/A'}</div>
            <div><strong>Canonical Unit:</strong> ${data.unit_canonical ||
'N/A'}</div>
            <div><strong>Parsed Range:</strong> ${data.range_low !== null ?
data.range_low : 'None'} - ${data.range_high !== null ? data.range_high : 'None'}</div>
            <div style="margin-top: 4px;">
                <strong>Analytics Flag:</strong>
                <span class="range-badge range-${
                    {'Within Range': 'within', 'Above Range': 'above',
'Below Range': 'below', 'Outlier': 'outlier', 'Invalid': 'invalid'}[data.test_analytics] ||
'invalid'
                }">${data.test_analytics}</span>
            </div>
        </div>
`;
} else {
    document.getElementById('ins-raw-values').innerHTML = `
        <div>
            <div><strong>Medicine:</strong> ${data.medication_medicine ||
data.medication_name || 'N/A'}</div>
            <div><strong>Dose:</strong> ${data.medication_dose || 'N/A'}
</div>
            <div><strong>Frequency:</strong> ${data.medication_frequency ||
'N/A'}</div>
        </div>
`;
    document.getElementById('ins-std-values').innerHTML = `
        <div>
            <div><strong>Canonical Name:</strong> ${data.medicine || 'N/A'}
</div>
            <div><strong>Dosage Form:</strong> ${data.medicine_type ||
'N/A'}</div>
            <div><strong>Dose:</strong> ${data.dose || 'N/A'}</div>
            <div><strong>Frequency:</strong> ${data.frequency || 'N/A'}
</div>
        </div>
    `;
}

```

```

        `;
    }

    // Confidence
    document.getElementById('ins-method').innerText = data.normalization_method
? data.normalization_method.replace('_', ' ').toUpperCase() : 'N/A';
    document.getElementById('ins-confidence').innerText =
data.normalization_confidence !== null ? `${Math.round(data.normalization_confidence *
100)}%` : 'N/A';

    // Raw JSON
    document.getElementById('ins-full-json').innerText = JSON.stringify(data,
null, 4);

    // Show modal
    document.getElementById('inspector-modal').style.display = 'flex';
    lucide.createIcons();
  })
  .catch(err => {
    alert("Failed to load record details.");
    console.error(err);
  });
}

function closeInspector() {
  document.getElementById('inspector-modal').style.display = 'none';
}
</script>
{% endblock %}

```